



**Politecnico
di Torino**

Politecnico di Torino

Corso di Laurea Magistrale

A.a. 2025/2026

Software Design and Architecture

Docenti:

Antonio Vetro'
Marco Torchiano

Studente:

Cristiano Berardo

Indice

1	Introduction to Software Design	1
1.1	Complexity	1
1.1.1	Syntax complexity	1
1.1.2	Cause of complexity	1
1.1.3	Strategical vs. Tactical Programming	1
1.2	Modules	1
1.2.1	Abstraction	2
1.2.2	Information Hiding	2
1.2.3	Information Leakage	2
1.2.4	Information Hiding Red Flags	2
1.2.5	Method separation	2
1.2.6	Do one simple thing and complexity	3
1.3	Obscurity	3
1.3.1	Abstraction layers	3
1.3.2	Writing Comments	3
1.3.3	Naming	3
1.4	Errors and Anomalies	4
1.4.1	Define Errors out of Existence	4
1.4.2	Limit exceptions	4
1.5	Dependency Dimensions	4
1.5.1	Structural (Code-Level) Dependencies	4
1.5.2	Runtime / Distribution Dependencies	5
1.5.3	Data-Level Dependencies	5
1.5.4	Behavioral Dependencies	5
2	Design Patterns	6
2.1	Types of Software Patterns	6
2.1.1	Architectural pattern	6
2.1.2	Design patterns	6
2.1.3	Idioms	7
2.2	Pattern language	7
2.3	Idioms	7
2.3.1	Strong typing vs. Duck Typing	7
2.3.2	Property	7
2.4	Design Patterns	8
2.4.1	Pattern selection	8
2.4.2	Creational patterns	8
2.4.3	Structural patterns	9
2.4.4	Behavioral patterns	11
2.4.5	Template Method	11
2.4.6	Strategy Pattern	12
2.4.7	Iterator pattern	12
2.4.8	Observer pattern	12
2.4.9	Visitor	13

2.5	Inheritance vs. composition	13
2.6	Abstract Syntax Tree	13
2.7	Model View Controller	13
3	What is software architecture ?	14
3.1	Goals of an architecture	14
3.2	The clear architectures	14
3.2.1	Entities	15
3.3	Use cases	15
3.4	Interface adapters	16
3.5	Frameworks and drivers	16
3.6	Separation of policy from details	16
4	From design to architecture: Design Principles	17
4.1	The SOLID design principles	17
4.1.1	Single Responsibility Principle - SRP	17
4.2	Open-Closed Principle - OCP	18
4.2.1	Liskov substitution principle - LSP	19
4.2.2	Interface segregation principle - ISP	19
4.2.3	Dependency inversion principle - DIP	20
4.2.4	DIP: practical implications	20
4.3	How to compile and see the UML	21
5	Components	22
5.1	Identify components	22
5.1.1	Putting components together	24
5.2	Component principles	24
5.2.1	Component cohesion	24
5.2.2	Component coupling	25
5.2.3	The Stable Dependencies Principle - SDP	26
5.2.4	Stable Abstractions Principle - SAP	27
5.3	Strategic decisions	27
5.3.1	Iterative Refinement	27
5.3.2	Strategic level: partitioning	28
5.3.3	Conway's law	28
5.3.4	Monolithic versus Distributed architecture	28
6	Architecture visualization: C4 notation	29
6.1	Visualizing software architecture	29
6.2	Issues in sw architecture visualization	29
6.3	C4 Model	29
6.3.1	Container	30
6.3.2	Component	30
6.3.3	Code	30
6.4	Different type of diagrams	30
6.5	Level 1: Context diagram	31
6.5.1	Information reported - People	31
6.5.2	Information reported - Software system	31
6.6	Level 2: Container diagram	31
6.6.1	Elements	32
6.6.2	Interactions between containers	32
6.7	Level 3: Components diagram	32
6.7.1	Elements	33
6.8	Level 4: Code diagram	34
6.9	Summary of C4 model	34

7	Wring Components	35
7.1	Components vs. Service	35
7.2	Framework	35
7.2.1	Inversion of control	35
7.2.2	Patterns that enable inversion of control	36
7.3	Wiring components	36
7.3.1	Interacction mode	36
7.3.2	Call binding	36
7.3.3	Method/funciton calls	36
7.4	Decoupling solutions	37
7.4.1	Dependency injection	37
7.4.2	Constructor injection	38
7.4.3	Setter injection	38
7.5	Service locator	39
7.5.1	Locator vs. Dependency injection	40
7.6	Messages/Events	41
7.6.1	Roles: Producer, Consumer	41
7.6.2	Roles: Broker	41
7.6.3	Delivery Guarantees	41
7.6.4	Structure	42
7.6.5	Shared state example	42
7.6.6	Effects	42
7.6.7	Challenges	43
7.7	Shared state	43
7.7.1	Effects	43
8	Communication	44
8.0.1	One-to-one communication	45
8.0.2	One-to-many communication	45
8.1	Messaging	45
8.1.1	Message types	45
8.2	Channel	45
8.2.1	Point-to-point channels	45
8.2.2	Publish-subscribe channels	46
8.3	Supported styles	46
8.4	Direct messaging - brokerless	46
8.5	Broker-based messaging	46
8.5.1	Broker characteristics	47
8.6	Coordination	47
8.6.1	Choreography	47
8.6.2	Orchestration	47
9	Microservices	48
9.0.1	Monolith vs. Microservices	49
9.0.2	Decomposition	49
9.0.3	Guidelines from OOP	50
9.1	Business logic	50
9.1.1	Transaction script	50
9.1.2	Domain model	51
9.1.3	Ports & Adapters (hex)	51
9.2	Domain Driven Design - DDD	51
9.2.1	Sub-Domain & Bounded Context	51
9.2.2	Class patterns	52
9.2.3	Domain boundaries	52
9.2.4	Aggregate pattern	52
9.2.5	Rule: reference roots only	52

9.2.6	Rule: primary keys inter aggregate	53
9.2.7	Rule: one aggregate per transaction	53
9.3	Split monolith with aggregates	53
9.3.1	Aggregate granularity	53
9.3.2	Domain Events	53
9.4	Communication	54
9.4.1	API evolution	54
9.5	Message formats	54
9.6	Remote procedure call (RPC)	54
9.6.1	Robust Synchronous APIs	55
9.6.2	Circuit Breaker pattern	55
9.6.3	Service Discovery	55
9.7	Messaging	56
9.7.1	Competing receivers	56
9.8	Shared channels	56
9.8.1	Duplicate messages	56
9.8.2	Transactional messaging	56
9.9	Data and Transactions	57
9.9.1	Database per service	57
9.9.2	Transactions in a monolith	57
9.9.3	Distributed data consistency	57
9.9.4	Saga pattern	57
9.9.5	Domain event pattern	58
9.9.6	Event sourcing	58
9.10	Queries	59
9.10.1	API Composition	59
9.10.2	CQRS	59
9.11	External API	60
9.12	Transition to Microservices	60
9.12.1	Strangling the monolith	60
9.12.2	Transition strategies	60
9.12.3	Refactor DB	60
9.12.4	God Class	61
10	Bending Spoons seminary (Evernote)	62
10.1	Evernote	62
10.1.1	Evernote's sync system	62
10.1.2	The sync service	63
11	Boundary	64
11.1	Independence and decoupling of layers	64
12	Monolithic Architectural Styles	65
12.1	Architectural Quanta	65
12.2	Architectural styles	65
12.3	Layered architecture - n tier	65
12.3.1	Layered architecture - variations	66
12.3.2	Adding layers	66
12.3.3	Isolation	66
12.3.4	Open layers	66
12.3.5	Open vs close layers	66
12.4	Modular monolithic style	67
12.4.1	Layered vs modular: components organization	68
12.4.2	Variations of modular monolithic style	68
12.4.3	Advantages and disadvantages of modular monolithic style	68
12.5	Pipeline architecture	69

12.5.1	Components	69
12.6	Micro-kernel - plug-in architecture style	70
12.6.1	UI: variants	70
12.6.2	Plug-in components	71
12.7	Monolithic Styles - recap	71

Capitolo 1

Introduction to Software Design

1.1 Complexity

Complexity is anything related to the structure of a software system that makes it hard to understand and modify the system.

$$C = \sum c_p t_p$$

Where C is determined by all software parts p , c_p is the complexity of part p , and t_p is the fraction of time spent on part p .

1.1.1 Syntax complexity

- Change amplification: change require operating in different places
- Cognitive load: how much information to learn to understand and perform a task. Some times is better to write longer code to reduce the complexity and so the cognitive load.
- Unknown unknowns: there are partes of the system that we don't know where they are or even that they exist.

1.1.2 Cause of complexity

Dependency: When we need to understand or modify a part of the code, we need to understand or modify other parts of the code. Try to reduce the number of dependencies and the complexity of the dependencies.

Obscuiry: alla the important information shoul be clearly visible.

1.1.3 Strategical vs. Tactical Programming

- **Tactical programming:** focus on having code that works as fast as possible. "Move fast and break things".
- **Strategical programming:** focus on long term maintainability and scalability. "Working code is not enough".

1.2 Modules

To improve and eliminate dependency and complexity, we can use modules. A module is a part of the code that has a well defined interface and a well defined implementation.

- **Interface:** what we need to know to use the module. Formal part, possible checked by the compiler. Represents a cost.

- **Implementation:** how the module works. Ideally much more complex than the interface. Represents a benefit.

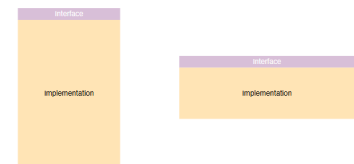
1.2.1 Abstraction

What we typically do is to perform an abstraction. Abstraction means selective removal of information. Is a simplified view of the something that omits "unimportant" information. Omitting information can lead to errors and keeping unimportant information can lead to cognitive overload.

Deep modules and shallow interfaces

Deep modules: modules with a simple interface and a complex implementation. They are good because they reduce the cognitive load and the change amplification. They are bad because they can lead to unknown unknowns.

Shallow interfaces: modules with a complex interface and a simple implementation. They are good because they reduce the unknown unknowns. They are bad because they can lead to cognitive overload and change amplification.



1.2.2 Information Hiding

Information hiding is the principle of hiding the implementation details of a module from the users of the module. Example sort function: we want to sort a list of numbers, we don't care about how the sorting algorithm used.

1.2.3 Information Leakage

Is the opposite of information hiding. Occurs when the implementation details of a module is reflected in multiple modules with consequences of creating dependencies.

1.2.4 Information Hiding Red Flags

- **temporal decomposition:** execution order is reflected in the code structure. Operations that happen in different times are in different classes.
- **overexposure:** API forces knowledge about rarely used features to use common ones. E.g.
`BufferedReader r = new BufferedReader(new InputStreamReader(System.in));`
- **Too many classes:** code is divided into too many shallow classes.

1.2.5 Method separation

Better together or Better apart?

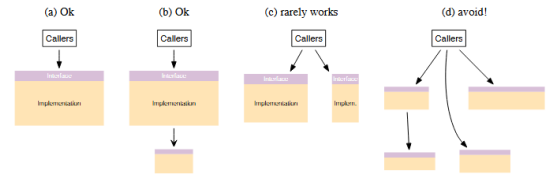
- **Bring together:** if information is shared, if we simplify the interface, if we eliminate duplication.
- **Separate:** if the code is general purpose or specific purpose.

Method separation red flags

If we have a lot of repeated code and/or we mixed specific and general code, means we have put together code that should be separated.

1.2.6 Do one simple thing and complexity

Each method should do one thing and do it completely



1.3 Obscurity

Obscurity is the principle of making the important information clearly visible and the unimportant information hidden. Obscurity is a great source of complexity. Obscure code is hard to Understand and creates more Bugs.

The solution to obscurity is writing obvious code, this means with a quick glance we can understand what the code does. We can achieve this by using abstraction layers.

1.3.1 Abstraction layers

Software systems are composed in layers. Higher layers use the facilities provided by lower layers.

Abstraction simplifies understanding and reduces obscurity.

In a well-designed system, each layer provides a different abstraction from the layers above and below it.

Abstraction Layers Red Flags

Some red flags that indicate that we have a problem with abstraction layers are:

- **Pass-Through methods:** is a method that does nothing but call another method.
- **Pass-Through variables:** is a variable that is passed through over and over down a long chain of method calls.

1.3.2 Writing Comments

Another method to reduce obscurity is writing comments. If users need to read a method code to use it, there is no abstraction.

Comments should describe things that are not obvious in the code:

- **Pick conventions:** write comments like other persons in the project
- **Don't repeat code:** avoid redundant comments that simply restate what the code already makes clear
- **Higher level comments enhance intuition**
- **Delayed comments are bad comments**
- **Use comments as design tool**

1.3.3 Naming

Toggether with comments, naming is a great tool to reduce obscurity. Bad names cause bugs. Name should create a image in mind of the reader about the nature of the thing being named. Names should be precise and consistent.

1.4 Errors and Anomalies

Exceptions add complexity because they create dependencies between the code and if they are easy to throw they are hard to handle.

A piece of code might throw exceptions in different ways:

- Caller may provide bad arguments or configuration
- Invoked method may not be able to complete the request
- In distributed systems network packets may be lost or delayed or peers may communicate in unexpected ways
- The code may detect bugs or internal inconsistencies or situations that are not prepared to handle

1.4.1 Define Errors out of Existence

What are the solutions to errors? Define errors out of existence. This means, not ignore the error but to redefine the behavior of a method so that no error is required.

E.g. `String.substring(int beginIndex, int endIndex)` it throws different exceptions in only specific cases. This is completely avoidable and unnecessary.

1.4.2 Limit exceptions

There are some methods to limit the number of exceptions that a method can throw:

- Handle exceptions at the point where they are thrown, if possible
- Exception aggregation means handle different exceptions with same handler
- Just crash
- Mask exceptions avoiding unexpected exceptions

1.5 Dependency Dimensions

1. Structural (Code-Level) Dependencies
2. Runtime / Distribution Dependencies
3. Data-Level Dependencies
4. Behavioral Dependencies

1.5.1 Structural (Code-Level) Dependencies

- **Implementation Dependency:** some piece of code depends on a concrete class instead of an abstraction
- **Construction Dependency:** you depend on how a collaboration, another class, is created.
- **Compile-Time Dependency:** the module knows something at compile time

1.5.2 Runtime / Distribution Dependencies

Often linked to distributed architectures.

- **Spatial Dependency:** A component depends on the location of another component. E.g. hard-coded URLs
- **Temporal Dependency:** A component depends on another being available at the same time. E.g. remote method calls or servers
- **Synchronization Dependency:** A component depends on another to complete work before continuing. E.g. synchronous request/response

1.5.3 Data-Level Dependencies

- **Data Dependency:** One component depends on another's internal data model. E.g. shared database
- **Schema Dependency:** Multiple services depend on the same database schema or representation structure. E.g. DB schema, XML/Json schema

1.5.4 Behavioral Dependencies

- **Control Flow Dependency:** One component dictates another's execution path. E.g. Centralized process managers
- **Transactional Dependency:** E.g. Components rely on atomic distributed consistency.
- **Timing Assumption Dependency:** Assumptions about response time or ordering. E.g. "Service must respond in <50ms" or Event ordering assumptions

Capitolo 2

Design Patterns

Initially proposed by Christopher Alexander in the context of architecture.

A reusable **solution**
to a known **problem**
in a well defined **context**

The context is a *design* situation giving rise to a (design) problem.

The problem is a set of forces repeatedly arising in the context.¹

The solution is a proven resolution of the problem. Configuration to balance forces. Structure with components and relationships and run-time behaviour.

Example of Social Pattern: You are in queue to the supermarket gastronomic section. The problem is hard to spot who arrived first. The solution is to use a ticket machine.

2.1 Types of Software Patterns

- **Architectural patterns** - high level structures of software systems. E.g., layered architecture, client-server, microservices.
- **Design patterns** - solutions to common design problems. E.g., Singleton, Observer, Factory Method.
- **Idioms** - language-specific patterns. E.g., RAII in C++, list comprehensions in Python.

2.1.1 Architectural pattern

Normally the architectural pattern works with big components and their interactions and is needed to describe the responsibilities of these components. Expresses a fundamental structural organization schema for software systems and defines the rules and guidelines for organizing the relationships between the components.

Example of architectural pattern: Several programs that are used in sequence read from input and write sequentially to output.

The problem is there are a lot of intermediate files used for communication between programs.

The solution is to adopt pipes and filters architecture feeding with the result of the previous one.

2.1.2 Design patterns

Provides a scheme for refining components of a software system or their relationships and describes a commonly recurring structure of communicating components.

¹Any relevant aspect of the problem E.g., requirements, constraints, desirable properties

Example of design pattern: A class library providing few functionalities contains a lot of classes. The problem is the user is exposed to the internal complexity of the library. The solution is to create a new façade class that interacts with the user and hide all the details.

2.1.3 Idioms

It is a low-level pattern specific to a programming language. Describes how to implement particular aspects of components or the relationships between them. Leverages the features of a programming language.

Example of idioms: An attribute is constant and should be globally available to many classes. The problem is opening access would allow unauthorized modifications and the attribute is repeated in every object. The solution is to make public static final in Java language.

2.2 Pattern language

Pattern do not exist in isolation in fact can happen that two or more patterns are applied together, used to implement part of another pattern and introduce a problem solved by another

They are called Pattern Languages or pattern systems.

Collection of patterns together with guidelines for: Should:

- Implementation
- Combination
- Practical use
- Count enough patterns
- Describe patterns uniformly
- Present relationships

2.3 Idioms

X-ability

Clonability, serializability, comparability, etc. some specific behaviour dealing with many heterogeneous classes. The solution is to derive from a class or interface that defines the required methods.

Example: In Java all methods extends the root class *Object* and provides the methods *toString()*, *equals()*, *hashCode()*, etc. that can be overridden to provide specific behaviour.

2.3.1 Strong typing vs. Duck Typing

"If it looks like a duck, swims like a duck, and quacks like a duck, then it probably is a duck."

Strong typed language: You must define interfaces and classes to be able to call a specific method. Compile time type checking.

Dynamic/duck typed language: You just add a method to a class and you can call it. No compile time type checking.

2.3.2 Property

A property, more general than an attribute, is a named characteristic of an object typically accessed through well-defined operations called accessors, getters, and setters forming the public interface of the object and it is not necessarily an attribute of the object (e.g. the age can be derived from the date of birth).

2.4 Design Patterns

Description of communicating objects and classes that are customized to solve a general design problem in a particular context.

A design pattern names, abstracts, and identifies the key aspects of a common design structure that make it useful for creating a reusable object- oriented design.

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	1	1	2
	Object	4	6	10

2.4.1 Pattern selection

How to use patterns? First of all you need to know the problem you are trying to solve, knowing the patterns and then select the one that best fits your needs.

2.4.2 Creational patterns

- Class

Factory Method

- Object

Abstract Factory

Builder

Prototype

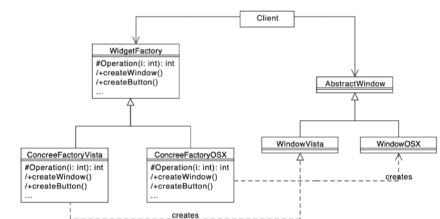
Singleton

Abstract Factory

A family of related classes can have different implementation details.

The client should not know anything about which variant they are using / creating.

Example: GUI where you have different components (buttons, text fields, etc.) and the same GUI can be applied to different platforms such as Windows, MacOS, Linux, etc.

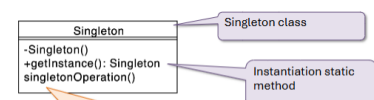


Singleton

A class represents a concept that requires a single instance.

The client should be able to access, in appropriate way, the unique instance.

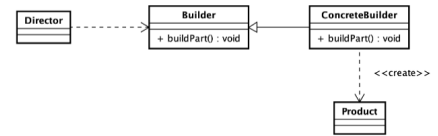
Note: If we talk about of an object designer pattern, in python you can just create a global variable and it will be a singleton.



```
private Singleton() { }
private static Singleton instance;
public static Singleton getInstance(){
    if(instance==null)
        instance = new Singleton();
    return instance;
}
```

Builder

An object of a complex class has to be created.
The creation entails complex interaction with the object and could be different variations of the object.



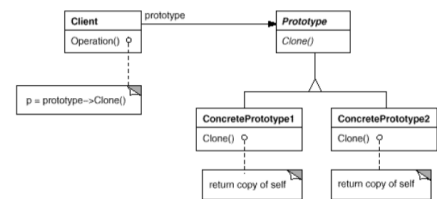
Example: $10.40 \text{ kg} \cdot \text{m}^2 \cdot \text{s}^{-3}$

```
//Usual non-fluent builder
Measure power = new Measure(10.4);
power.addUnit("kg", 1);
power.addUnit("m", 2);
power.addUnit("s", -3);
power.setPrecision(2);
```

```
//Fluent builder
Measure power = new Measure.Builder(10.4)
    .is("kg")
    .by("m")
    .squared()
    .by("s")
    .to(-3)
    .withPrecision(2)
    .build();
```

Prototype

Several object can be created out of a palette of similar classes.
The class hierarchy can become quite complex and classes in hierarchy are differentiated by minor parameters (that could be defined through constructor arguments).
A solution could be start with a prototype classe and clone it to create new objects.



Example: A graphical editor where you have very similar shapes (e.g. square with different radius, colors, etc.).

2.4.3 Structural patterns

How organize classes and objects to compose larger structures.

- Class

Adapter

- Object

Bridge

Composite

Decorator

Facade

Flyweight

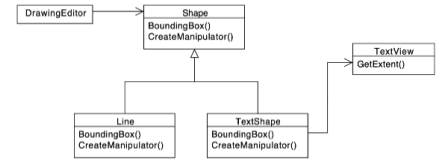
Proxy

Adapter

A class provides the features required by another class but its interface is not the one expected by the client.

The integration of the provider class should be possible without modifying it. The source code could be not available or is already used as it is somewhere else.

Example: In Java, keyboard events are handled by `KeyListener`. Listener class need to implement the interface and implement all the methods. If you are interested only in one of the methods, you can use the `KeyAdapter` class that is empty and you can decide which method to override, and the Listener extends `KeyAdapter` instead of `KeyListener`.

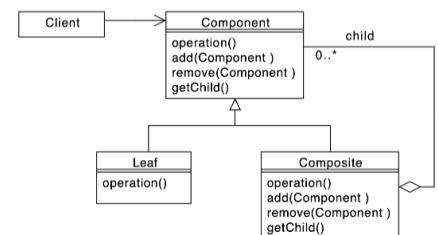


Composite

You need to represent part-whole hierarchies of objects.

Clients can be very complex and they require to adapt the difference between all the elements in the hierarchy.

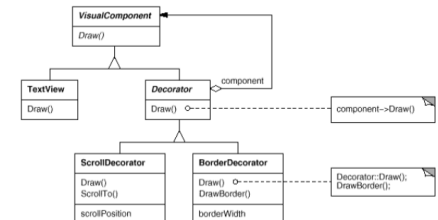
Example: In Java UI components are organized in a tree structure. You can have a panel that contains other panels and buttons, etc.



Decorator

You have some objects of classes that have specific behaviour and you want to add the same behaviour plus something else dynamically.

The solution can be extend the class but if that behaviour is something you would to apply to many classes, you need to extend all of leaf classes. Using a decorator you can modify the behaviour without creating an explosion of subclasses for every combination of features

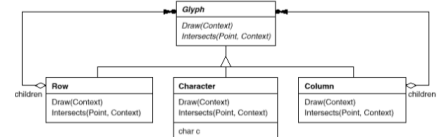


Example: In Java we have `JScrollPane` can be used to provide a scroll pane to any component

Flyweight

You need need to create a very large number of similar objects, and memory usage and performance become concerns.

Example: In Java we can pool different objects together. The `Integer` class in Java.



Proxy

You need to controll access to an object.

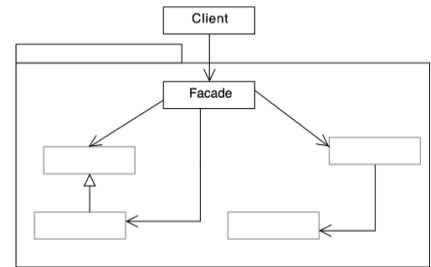
We need to provide a placeholder that is able to mediate between the client needs and all the requirements you want to controll before accessing the final object without changing interface or class.

- **Remote proxy:** provides a local representative for an object in a different address space.
- **Virtual proxy:** creates expensive objects on demand. E.g. slow loading resource - images in a web page.

- **Protection proxy:** controls access to the original object. E.g. when objects should have different access rights.
- **Smart reference:** is a replacement for a bare pointer that performs additional actions when an object is accessed. E.g. counting the number of references, checking permissions, etc.

Facade

Hide the details and the complexity of a general class by providing a simpler interface. The client should not know anything about the subsystem.



2.4.4 Behavioral patterns

Behavioral patterns allow to implement specific algorithms using objects and classes to provide control and flexibility.

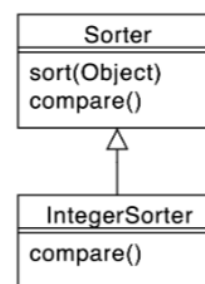
- Object
 - Chain of Responsibility
 - Command
 - Iterator
 - Mediator
 - Memento
 - Observer
 - State
 - Strategy
 - Visitor
- Class
 - Interpreter
 - Template Method

The basic mechanism of behavioral patterns is enapsultion variation. Instead of having a lot of if-else you have a structure at object level that allows you to define different alternatives.

2.4.5 Template Method

An algorithm/behavior has a stable core and several variation at given points. You have to implement/maintain several almost identical pieces of code.

Example: The compare in Java. You have to implement the compare method that solves integer or string comparison.



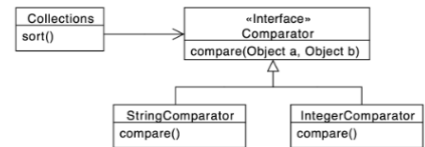
2.4.6 Strategy Pattern

Many classes or algorithm has a stable core and several behavioral variations, several different implementations are needed. Every variation of an algorithm can be embedded in an object and passed as a parameter to the algorithm.

Example: Comparator in Java.

Consequences:

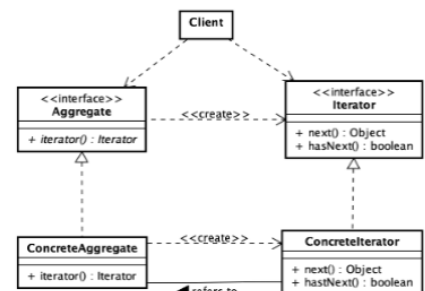
- + Avoid conditional statements
- + Algorithms may be organized in families
 - + Choice of implementations
 - + Run-time binding
- Clients must be aware of different strategies
 - Communication overhead
 - Increased number of objects



2.4.7 Iterator pattern

You have a collection of objects and you want to iterate on it allowing multiple concurrent iterators.

Example: In Java there is the Iterable interface and Iterator.



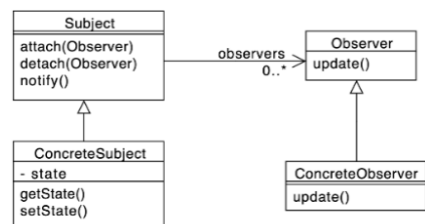
2.4.8 Observer pattern

The change in one object may influence one or more other objects. This can cause high coupling between objects and the number and type of objects to be notified may not be known in advance.

Example: In Java there is the Observable interface and Observer interface. This is very common in UI programming.

Consequences:

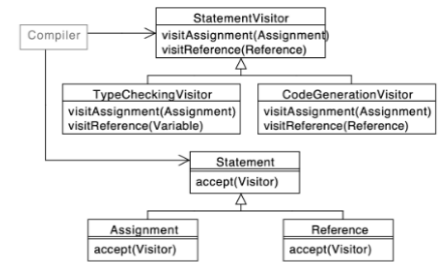
- + Abstract coupling between Subject and Observer
- + Support for broadcast communication
 - Unanticipated updates



2.4.9 Visitor

You have an object structure that contains many classes with different interfaces and you want to perform several different and unrelated operations on these objects.

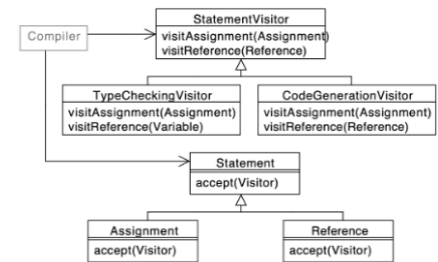
The operations depend on the concrete classes and you want to avoid putting many different methods in each of these classes.



2.5 Inheritance vs. composition

Reuse can be achieved via:

- **Inheritance:** a new class is created by inheriting from an existing class and adding new features or modifying existing ones. Clients can invoke directly inherited methods.
- **Composition:** The class has a link or instance to a class you want to reuse and just forwards the calls to it.



Note: The Observable pattern is deprecated. The PropertyChangeSupport is more flexible.

2.6 Abstract Syntax Tree

The AST is a tree that represent a syntax of formal language. We can use the composite pattern to representation the arithmetic expressions.

And using the visitor pattern or the interpreter pattern to evaluate the expression.

2.7 Model View Controller

Capitolo 3

What is software architecture ?

The architecture of a software system is the shape given to that system by those who build it. The form of that shape is in the division of that system into components, the arrangement of those components, and the ways in which those components communicate with each other. The purpose of that shape is to facilitate the development, deployment, operation, and maintenance of the software system contained within it.

3.1 Goals of an architecture

The software must change to adapt to software regulation, user feedback.

We have some pattern that make methods and classes easy to be changed, but now we want something behind the methods.

The goals of an architecture are:

- keep the system valuable for its users over time
- minimize the lifetime cost of the system
- maximize programmer productivity

This can be achieved by making the system:

- easy to understand
- easy to develop
- easy to maintain
- easy to deploy
- easy to change

In Java we have the package, that is a module, and is a logical grouping of classes.

Design Patterns: helps to build methods and classes inside the module in a way that they are easy to be changed.

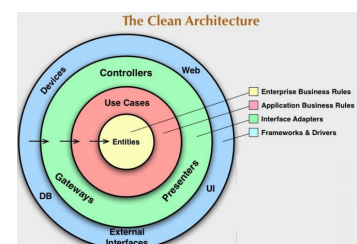
Design Principles: define the modules can collaborate together and how they are shaped.

3.2 The clear architectures

The **Outer circles:** mechanisms, they change often.

The **Inner circles:** policies, subject to less frequent changes.

Source code dependencies point only inward, toward higher-level policies (Dependency Rule)



3.2.1 Entities

An Entity is an object within a software system that contains a set of critical business rules operating on Critical Business Data.

The interface of the Entity consists of the functions that implement the Critical Business Rules to operate on the Critical Business Data. They contain the most general and highlevel rules and neither they are often changed nor they are usually affected by lower-level changes.

Example: The system is the university portal and an important entity is the student, that has some critical business rules like "the student can enroll to the master degree only if they have completed the required prerequisites and then assign new ID".

Is the most stable part of the system and we want to preserve as much as possible from changes occurring to the other elements.

Student
- ID - DateBirth - Address
+ assignGrade() + assignSubsidie() + computeTax()

3.3 Use cases

To apply the Business Roles to the entities we need to use the use cases.

A use case is a description of the software behavior and interactions (with people or other software) in a specific case.

It specifies:

- the input to be provided by the user (or other systems)
- the output to be returned
- The processing steps involved.

to determine the flow of data to and from the entities.

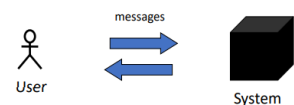
Software at this level change whenever a change to any element of a use case is required

Use case in practice

It represents a **functional requirement**.

- It makes clear what you expect from a system ("what?")
- It hides the behaviour of the system ("how?")
- It is a sequence of actions (with variants) that produce a result observable by an actor

The system is seen as a black-box. Use cases express the interaction (exchange of messages) between the entities that interact with the system itself (called actors).



Example - Specify annual tax for a student Input: student ID

Output: annual tax amount

Main scenario:

1. Validate ID provided by user
2. Retrieve economic level
3. Compute due past taxes
4. Compute new taxes
5. Get average grades for the last academic year
6. If average grade > 26, apply 20% discount
7. Ask operator for confirmation: if yes change student else reset

3.4 Interface adapters

Their goal is to shape data in a way that it is convenient to higher levels or other systems or artifacts interacting with:

The same is done in the opposite direction. Examples of code in this layer: Presenters for GUI, Controller for DB

Code inward to this level know anything about details (e.g., SQL)

3.5 Frameworks and drivers

In order to achieve the separation we have seen so far, we need to separate low level details from the high-level details. Layers allow independent development, deployment, operation, and maintenance.

The details are the frameworks, the database, the drivers, the web server, etc., and usually we do not write much code here.

3.6 Separation of policy from details

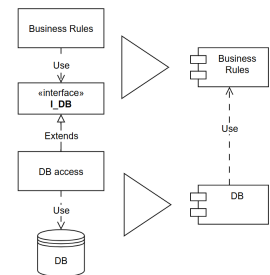
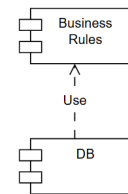
Decoupling of policy from the details and defer decisions on details for as long as possible (MongoDB or SQL, etc.)

DB component knows about use case rules component, but NOT viceversa.

Low level components know everything of the higher-level components, but NOT viceversa

The Database component contains the code that translates the calls made by the BusinessRules into the query language of the database. This translation code knows about the BusinessRules.

Business rules can be adapted to any kind of DB.



Capitolo 4

From design to architecture: Design Principles

Design principles means follow a clean architecture and a clean architecture means that the architecture can easily evolve, to be changed.

4.1 The SOLID design principles

The SOLID principles suggest software architects on how to arrange functions and data into groupings (e.g., classes), and how these groupings are arranged.

The goal of the SOLID principles is to create mid-level software structures (modules) that:

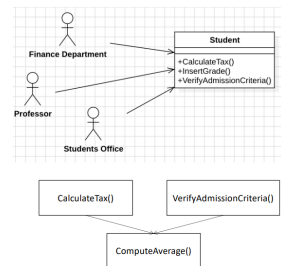
- Are easy to understand
- Allow changes in the most "comfortable" way
- Are the basis of reusable components

4.1.1 Single Responsibility Principle - SRP

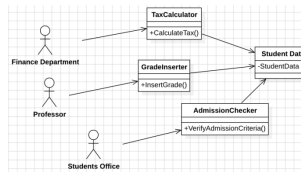
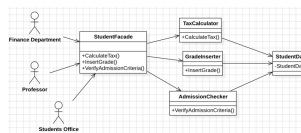
A module should have one, and only one, reason to change. It means that a module should be responsible to **one**, and only one, **actor**.

This is helpful to separate code used by different actors and for a better maintenance.

Example *CalculateTax()* is specified by the Finance Department.
InsertGrade() is specified by Students Office and used by Professor
VerifyAdmissionCriteria() is specified and used by Students Office
CalculateTax() and *VerifyAdmission()* share a common algorithm for calculating the average grade.

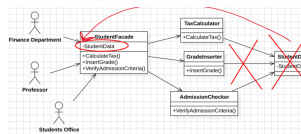


Changes in *ComputeAverage()* affect two actors: errors may arise and also changes to Class Student due to different needs of actors may lead to collisions and merges.

Solution 1: Separate data from functions**Solution 2: Use the Façade pattern**

You increase coupling and complexity, its a trade off, but the separation is splitted.

Note: You increase coupling and complexity but the separation is splitted.

Solution 2: variation**4.2 Open-Closed Principle - OCP**

A software artifact should be **open for extension but closed for modification**. Using design patterns to make new functionality in a way that is additive and not to modify the existing structure.

The behavior of a software artifact ought to be extendible, without having to modify that artifact.

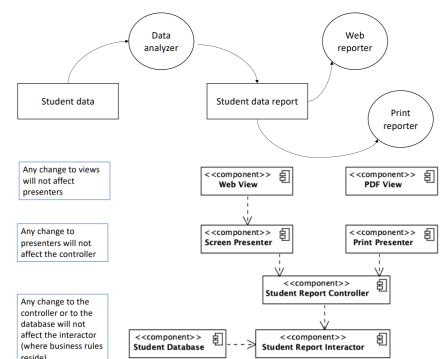
The aim of OCP is to make the system easy to extend without requiring too many changes to the system.

Using the open-closed principle, the software architect divides the system into components and arranges these components in a dependency hierarchy that protects higher-level components from changes in lower-level components.

Example A system displays summary information about a student's career at the university on a dashboard: Subjects (those passed with grades), Taxes paid and due, Current semester schedule, Next exams, Important notices, Stakeholders ask for a printed report with only information on subjects and taxes, and a different formatting.

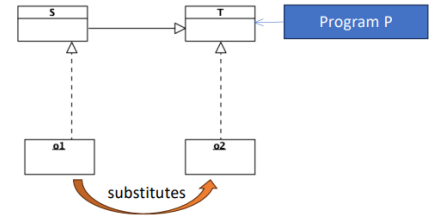
Generating the report involves two separate responsibilities:

1. the calculation of the reported data
2. the presentation of that data into a web- and printer-friendly form.



4.2.1 Liskov substitution principle - LSP

If for each object *o1* of type *S* there is an object *o2* of type *T* such that for all programs *P* defined in terms of *T*, the behavior of *P* is unchanged when *o1* is substituted for *o2* then ***S* is a subtype of *T***.

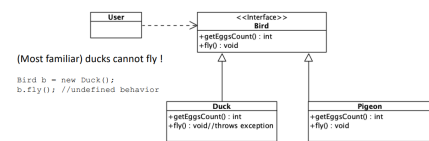


If we substitute *o2* with *o1* and the behavior of *P* does not change means *S* is a derived class of *T*.

This principle is about subtyping, not only for inheritance or extension but also for the logic, the behavior of the class.

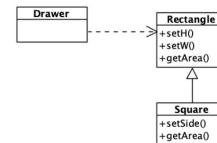
Derived type must fully support the substitution of their base types: functions that use pointers or references to base classes must be able to use objects of the derived without knowing it. This is related to the substitution property.

A subtype can replace its supertype only if it behaves the same way. This idea, called **behavioral subtyping**, means the subtype doesn't just have the same methods as the supertype — it also follows the same rules for how those methods should work. That way, anything the client expects from the supertype will also hold true for the subtype.



```
Rectangle r = ...;

r.setW(5); r.setH(2);
assert(r.area() == 10);
```



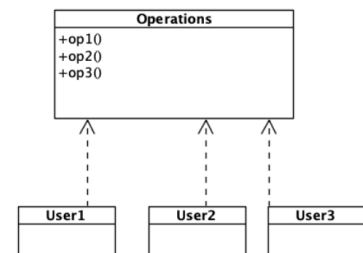
Depending on how *r* is defined, the assertion can fail because you not use the *setH()* and *setW()* methods of the rectangle but you use the *setSide()* method of the square.

How to refactor it? Using the Interface segregation principle - ISP

4.2.2 Interface segregation principle - ISP

Clients should not be forced to depend on operations and/or interfaces they do not use. Better to make fine grained interfaces that are client specific.

Similar to SRP we have different actors, represented by different classes. They are only interested in a subset of the operations, so we can split the interface in U1 use only operation1 and U2 use only operation2 and U3 use only operation3



In a statically typed language (e.g., Java), any change to *op1()* will force recompilation and redeployment of *User2* and *User3*.

Is different from the SRP because we need the classes but we need to modularize better.

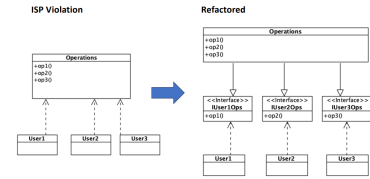
Inserting interfaces the classes use the interfaces instead the class and the interface implement the class.

If we change operation1 we impact also user2 and user3, if we change the interface we impact only the corresponded user.

We also package the interface and the user together and this not impact the module.

Operations are segregated into interfaces.

In a statically typed language (e.g., Java), a change to *op1()* will NOT force recompilation and redeployment of User2 and User3.



Remove useless operations from modules, otherwise they will force useless redeployments

4.2.3 Dependency inversion principle - DIP

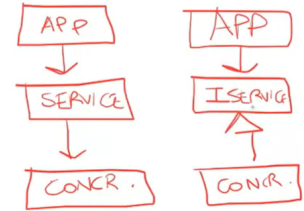
You should always have dependencies that lower modules depend on higher modules, not the opposite.

High-level modules should not depend on anything from low-level modules.

Stable abstractions: abstractions should not depend on details, so that changes to details will not affect them.

Idealistic tension, in practice each system has at least a concrete component: the main function

We can make an interface:



Example In Java (or other statically typed languages), this means that use, import and include statements should refer only to source modules containing interfaces, abstract classes or any other type of abstraction.

In dynamically typed languages, source code dependencies should not refer to modules in which functions being called are implemented.

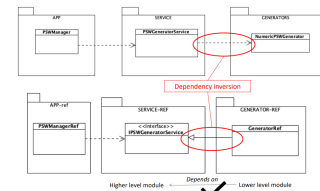
Also we can package the app and the interface together.

4.2.4 DIP: practical implications

Dependencies on concretions are tolerable when it comes to operating systems, platform facilities and other similar components whose changes are tightly controlled and not frequent.

Abstract Factories are helpful to adhere to the DIP by minimizing the number of dependencies on concrete implementations and keep them separate from the rest of the system

In general, volatility should be shifted towards implementation details without changing interfaces.

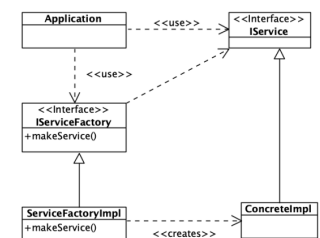


- Don't refer to volatile concrete classes
- Don't refer from volatile concrete classes
- Don't override concreted functions
- Never mentions the name of anything concrete and volatile

Using Factory Pattern

To create an instance of ConcreteImpl, the Application calls *makeService()* from IServiceFactory interface, which is implemented by ServiceFactoryImpl class.

ConcreteImpl is created and returned as a IService.



Separation abstract and concrete levels: All source code dependencies point toward the abstract level. The flow of control crosses the separation line in the opposite direction: this is the sense of "dependency inversion".

DIP violation DIP violations cannot be entirely removed. Some dependencies on concrete implementations are necessary. They must be minimized and kept separate

4.3 How to compile and see the UML

```
mvn -f .\Path\to\pom_file\pom.xml clean compile site
```

Then Java Project > Target > site > index.html

Capitolo 5

Components

The logical components are the building blocks of the system. They usually manifest through a namespace or a directory structure, where leaf nodes containing source code represent the logical components.

Logical architecture: the system's logical components and how they interact each other.

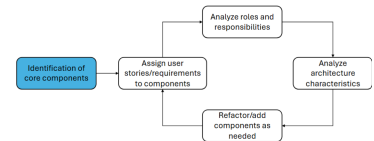
Physical architecture: includes also physical artifacts, defining architectural

5.1 Identify components

Example: SPG = Solidarity Purchasing Group, Solidarity towards producers valuing (small vs large, local and seasonal vs global and specific vs mass).

Solidarity among clients: Not a normal shop but a group of people buying together, sharing the principles to select producers.

This is a software application for **Clients**, **Farmers** and **Shop employees**.



The identification of components is fundamental activity, necessary either for drafting architecture from scratch or reverse-engineer it. It is an iterative refinement process and three approaches can be used for initial identification:

- **Best guess:** based on the system core functionalities
- **Workflow:** Based on the "happy path"
- **Actor/action:** Based on major actions a user can perform

SPG requirements

A neighborhood association manages a Solidarity Purchasing Group (SPG). Every Monday, a farm (rotating among a pool of reference suppliers) submits to the SPG a list of available products, their quantities, and their prices.

Once this list is provided, the data is reviewed and confirmed by the SPG employee.

SPG members receive an email with the list of available products and can place their orders either via the website or by visiting the association's headquarters.

When ordering, SPG members specify the types of vegetables they want and the quantities (expressed in kilos, packs, or boxes depending on the type of fruit or vegetable ordered). If the order is placed at the association's headquarters, the member pays immediately.

At the end of the week, the farm delivers the products to the association's headquarters. Shop employees record the delivery, and members receive a notification when their ordered goods arrive. At this point, members can go to the association to collect the package (paying at the same time, if not done at the time of ordering).

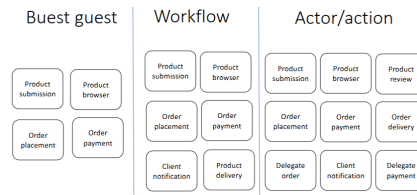
Macro functionality	Component
Insert products	Product submission
View products	Product browser
Order products	Order placement
Buy products	Order Payment

Workflow approach

Step	Component
1. Farmer submits available products, employee reviews	Product submission
2. SPG members/clients receive list of products	Client notification
3. Client browse the catalog of product	Product browser
4. Client places an order	Order placement
4a. Employee places an order for the client	Order placement
5. Client pays the order online	Order Payment
5a. Employee submit order payment	Order Payment
6. Employees register delivery of goods	Product delivery
7. Clients are updated on goods' delivery	Client notification
8. Clients retrieve goods	Product delivery

Actor/action

Actor	Action	Component
Client	Search for products	Product browser
	View details about products	Product browser
	Place order	Order placement
	Pay order	Order payment
Farmer	Insert products availabilities	Product submission
Shop employee	Review products availabilities	Product review
	Insert client order	Delegate order
	Insert client payment	Delegate payment
	Register product delivery	Order delivery
System !	Send email notifications	Email Notification



5.1.1 Putting components together

- **Tactic level - design:** Components principles: cohesion, coupling
- **Strategic level - architecture:** Component topology, technical partitioning vs domain partitioning, and Physical architecture, Monolithic vs Distributed architecture.
- **And more:** Deployment, communication style and data topology

5.2 Component principles

5.2.1 Component cohesion

How to put classes inside a component?

Cohesion is the degree to which the classes within a component belong together.

High cohesion means that the classes within a component are closely related in terms of functionality and purpose, while **low cohesion** indicates that the classes are more unrelated and serve different purposes.

It is suggested to follow principle:

- The **Reuse/Release Equivalence Principle - REP:** The granule of reuse is the granule of release.
- The **Common Closure Principle - CCP:** Classes that change together are packaged together.
- The **Common Reuse Principle - CRP:** Classes that are used together are packaged together.

Reuse/Release Equivalence Principle - REP

To support reuse, classes and modules that make up a component must: belong to a coherent group and be released together.

The release process shall produce appropriate notifications and release documentation, so that users can decide whether to integrate new releases.

Common Closure Principle - CCP

Group into components classes that change together, i.e. for the same reason and at the same time.

Separate into different components classes that have different changes processes and motivation.

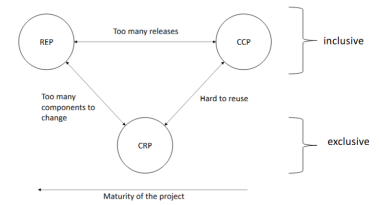
Note: There is a relationship between the Common Closure Principle (CCP). It is similar to SRP and it addresses the "closure" aspect of OCP

Common Reuse Principle - CRP

It help deciding which classes/module to place into the same components. In such components, usually classes have many dependencies on each other

Complementary perspective:

- users of a component should not depend on unnecessary features;
- if a class isn't closely connected to another class, it shouldn't be in the same component.



Similar to interface segregation design principle (ISP).

Note: Similar to interface segregation design principle (ISP)

5.2.2 Component coupling

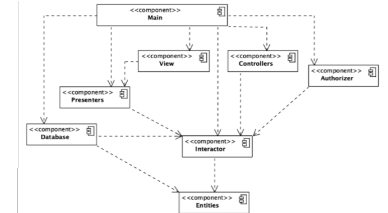
- The Acyclic Dependencies Principle (ADP): The dependency graph of packages must have no cycles.
- The Stable Dependencies Principle (SDP): Depend in the direction of stability.
- The Stable Abstractions Principle (SAP): Abstractness increases with stability.

Acyclic Dependencies Principle - ADP

Historical roots on typical issues of team-based development: the *morning after syndrome*¹ and the *weekly build*.

Solution to the problem: partition the development environment into releasable components

Each component has its own responsible developer/team so the releases are uniquely identified and components follow their own, private, development path. E.g. in git versioning system, using separated branches or even separated repositories.



- **Advantages:** Changes made to one component do not affect other teams, until they decide to adapt to the new component's release. Integration happens in small steps
- **Implications:** teams must monitor, manage and, maintain the dependency structure over time. there can be no "cycles" or circular dependencies in the dependency graph.

Example: directed acyclic graph Starting at any component, it is not possible to get back to the origin by following the edges (dependencies).

What happens when a change is made to Presenters? View and Main will be affected and tests with Interactor and Entities.

The bottom-up "building" for system release

- Entities
- Database and Interactor
- Presenters, View, Controllers, Authorizer
- Main

¹The morning after syndrome occurs when code worked on, tested, and left working one day is broken the next morning due to changes made by others

Example 2: dependency cycle Now imagine a new dependency arises: User class in Entities uses the Permissions class in Authorizer.

Testing Entities requires now also Authorizer and Interactor.

There are different ways to solve this problem:

- Solution A: Use dependency inversion principle (DIP)
- Solution B: Create a new component on which both Entities and Authorizer depend. Move the class(es) they both depend on into this new component.

5.2.3 The Stable Dependencies Principle - SDP

Depend in the direction of stability. Stability is not about the frequency of changes, but about the amount of work required to make a change.

A cupboard is stable because it takes a lot of effort to move it, while a cup is not stable because it takes little effort to move it.

A component with lots of incoming dependencies is very stable because it requires a great deal of work to reconcile any changes with all the dependent components.

Modules that are designed to be harder to modify should not depend on modules that require little effort to change.

A simple measure of (in)stability

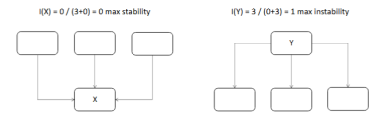
X is very stable because three components depend on it and it depends on no other component, while Y is very unstable because no component depends on it and it depends on three other components.

Instability: goes from 0 to 1, where 0 is the most stable and 1 is the most unstable.

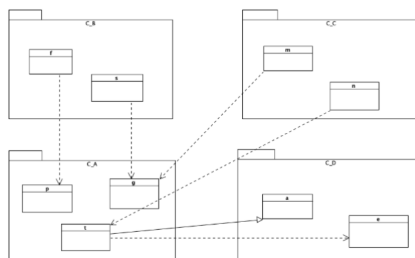
$$I = \frac{\text{Fan-out}}{\text{Fan-in} + \text{Fan-out}}$$

Where

- Fan-in is the number of incoming dependencies
- Fan-out is the number of outgoing dependencies



Example: $I(C_B) = 0.33$. Fan-in = 4 and Fan-out = 2. $I(C_D) = \frac{2}{4+2} = 0.33$.

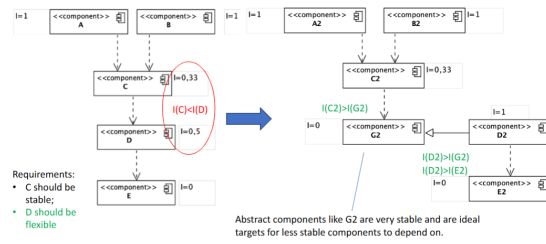


SDP and stability measure

Depend in the direction of stability.

The $I(\text{component})$ should be larger than the $I(\text{the components that it depends on})$. I metric should decrease in the direction of dependency. $I(C_B) = I(C_C) > I(C_A) > I(C_D)$

SDP violation as system evolves: example Requirements: C should be stable. D should be flexible.



5.2.4 Stable Abstractions Principle - SAP

A component should be as abstract as it is stable. Code that represent high-level policies of the system should be placed into stable components.

SAP, OCP, SDP and abstract classes

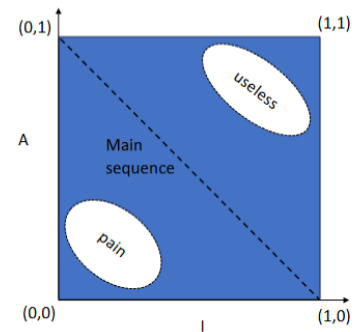
Open Closed principle suggests creating classes that are flexible enough to be extended without requiring modification. Abstract classes are best suited to this need. Thus, applying SDP: stability goes into the direction of abstraction

A simple measure of abstractness this ratio:

$$A = \frac{\text{Number of abstract classes and interfaces in the component}}{\text{Total number of classes in the component}}$$

where:

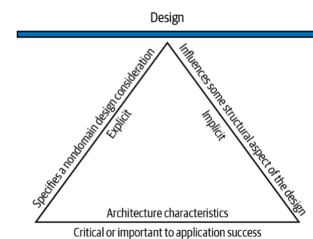
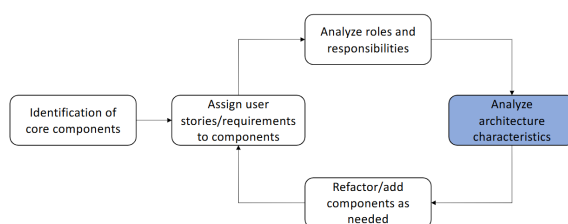
- $A = 0$ means that the component is completely concrete. No abstract classes.
- $A = 1$ means that the component is completely abstract. Only abstract classes.



Note: Distance from the main sequence: $D = |A + I - 1|$

5.3 Strategic decisions

5.3.1 Iterative Refinement



Operational Architectural Characteristics: examples

- **Scalability:** the system's ability to perform and operate as the number of users or requests increases
- **Elasticity:** the system's ability to perform and operate with peaks of requests
- **Responsiveness:** the system's ability to complete assigned tasks within a given time
- **Reliability/safety:** Whether the system needs to be fail-safe, or if it is mission critical in a way that affects lives. Fault tolerance does the software operate as intended despite hardware or software faults.

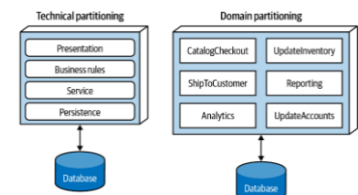
- **Availability:** How much of the time the system will need to be available; if that's 24/7, steps need to be in place to allow the system to be up and running quickly in case of any failure.
- **Continuity:** The system's disaster recovery capability.
- **Robustness:** The system's ability to handle error and boundary conditions while running, for example, if the internet connection or power fails.
- ...

Engineering Architectural Characteristics

- **Maintainability:** the degree of effectiveness and efficiency to which developers can modify the software to improve it, correct it, or adapt it to changes in environment and/or requirements.
- **Testability:** how easily developers and others can test the software? How complete are tests?
- **Deployability:** ease, frequency and overall risk of deployment
- **Evolvability:** system's ability to survive changes in its environment, requirements and implementation technologies
- ...

5.3.2 Strategic level: partitioning

Technical partitioning vs domain partitioning



5.3.3 Conway's law

Organizations which design systems...are constrained to produce designs which are copies of the communication structures of these organizations.

The structure of the design will reflect how people communicate and work together

5.3.4 Monolithic versus Distributed architecture

Monolithic: single deployment unit of all code

Distributed: multiple deployment units connected through remote access protocols

Monolithic	Distributed
Layered architecture	Service-based
Pipeline architecture	Event-driven
Modular monolith	Space-based
Microkernel	Service-oriented
	Microservices

Capitolo 6

Architecture visualization: C4 notation

6.1 Visualizing software architecture

There are not only one diagrams to visualize software architecture, but many of them.
Variety of goals, stakeholders, focus, etc.

6.2 Issues in sw architecture visualization

- Purpose of elements (boxes, arrows, etc.)
- Relationship between elements
- Levels of abstraction
- No context or logical starting point
- Consistency between diagrams
- Require explanations or descriptions to be fully understood

Diagrams are like maps: different providers, google or openstreetmap have different purposes. For this reason, we will introduce the C4 model, which is a standard for software architecture visualization.

6.3 C4 Model

With this model we can zoom-in and zoom-out to navigate through the different levels of abstraction to use with different stakeholders.

A **software system** is made up of one or more **containers**, each of which contains one or more **components**, which in turn are implemented by one or more code elements. And **people** use the software systems that we build.

It is organized in 4 levels of abstraction:

- Software System
- Container
- Component
- Code Element

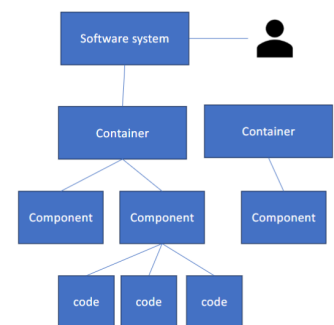


Figura 6.1: Static structure

6.3.1 Container

A boundary inside which some code is executed or some data is stored. Containers are **separately deployable**.

Examples:

- Application (server-side or client side)
- Database, file system, etc.
- Microservice (if not a system per se)
- Mobile app
- Script (python, shell, R, etc.)

6.3.2 Component

A grouping of related functionality encapsulated behind a well-defined interface

Components are **NOT** separately deployable and how components are packaged is not relevant at this level.

6.3.3 Code

The basic building blocks of a programming language:

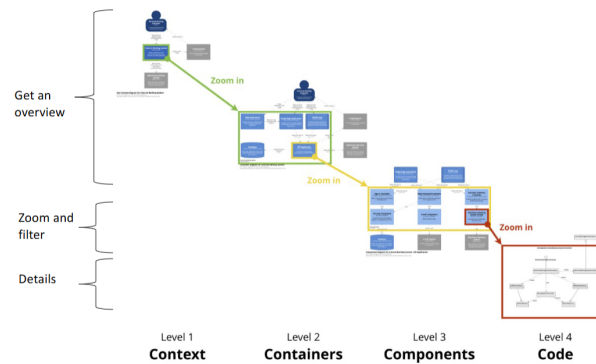
- classes,
- interfaces,
- functions,
- objects,
- enums

6.4 Different type of diagrams

The C4 model defines different types of diagrams for each level of abstraction.

- **Context diagram:** it shows how the software system in scope fits into the context around it.
- **Container diagram:** it shows the high-level technical building blocks (containers) and how they interact.
- **Component diagram:** it shows the components inside each container
- **Code diagram:** it shows how a component is implemented

Using the SPG, Solidarity Purchasing Group, presented before, as an example, we can create different diagrams for each level of abstraction.



6.5 Level 1: Context diagram

It helps answering such questions:

- What is the software system that we are building (or have built)?
- Who is using it?
- How does it fit in with the existing environment?

It is **required**: it is the starting point for communicating with any stakeholder
The key elements of a context diagram are:

- People who will use the software system
- Other software systems with which the reference system interacts
- + optionally the organization boundary

Interaction between elements: Directed arrows with high-level information about the purpose of that interaction. We can connect elements with arrows. It is better to use only one way arrows and not to use bidirectional once.

6.5.1 Information reported - People

The people element have:

- **Name:** the name of the person, user, role, actor or persona.
- **Description:** A short description of the person, their role, responsibilities, etc



6.5.2 Information reported - Software system

The software system element have:

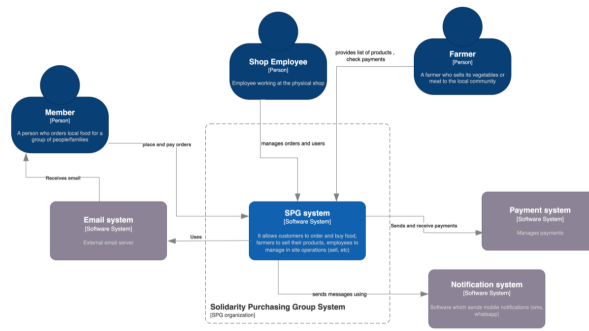
- **Name:** The name of the software system.
- **Description:** A short description of the software system, its goal and responsibilities, etc.



6.6 Level 2: Container diagram

Goal: illustrate the high-level technology elements and how responsibilities are distributed across elements and how containers communicate with each other

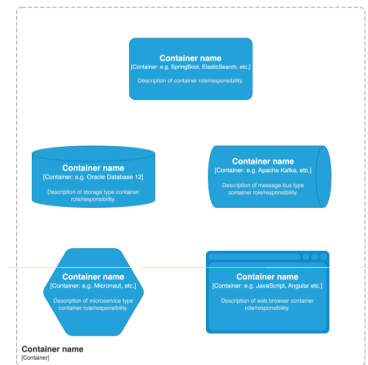
Every container must be deployable independently and the diagram is **required** and readed by software development adn operations stuff.



6.6.1 Elements

People, software systems and containers. For each container:

- Name: The name of the container (e.g. “Internet-facing web server”, “Database”, etc).
- Technology: The implementation technology (e.g. Java/Spring MVC application, ASP.NET web application, Relational Database Schema, etc.) or the general type of technology”
- Description: A short descriptive statement, or perhaps a list of the container’s key responsibilities or entities/tables/files/etc. that are being stored.



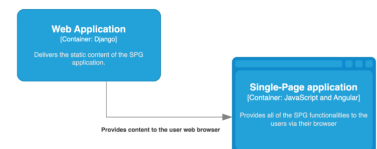
System boundary must be coherent with L1

6.6.2 Interactions between containers

Interactions between containers must be as explicit as possible.

Example of useful information to annotate the interactions with:

- The purpose of the interaction (e.g. “reads/writes data from”, “sends reports to”, etc).
- The communication mechanism (e.g. REST, Web API, Java Message Service, Web Services).
- The communication style (e.g. synchronous, asynchronous, batched, twophase commit, etc).
- Protocols and port numbers (e.g. HTTP, HTTPS, SOAP/HTTP, SMTP, FTP, RMI/IIOP, etc)

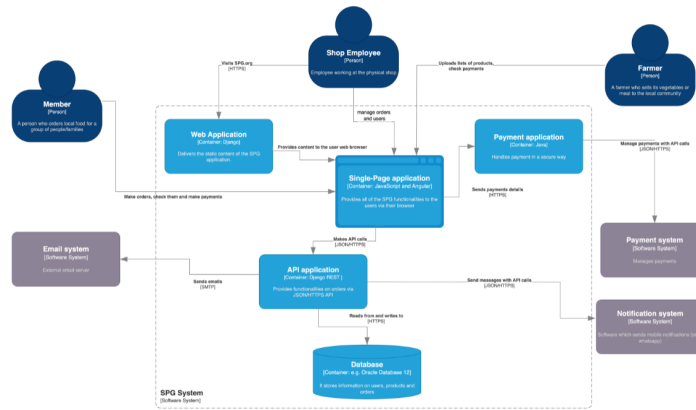


6.7 Level 3: Components diagram

It shows components that are within a single container at a time. Infrastructural elements (e.g. , logging, metering) and shared library might be annotated, separated, or excluded.

It helps to answer the questions like

- What components is each container made up of?
- Do all components stay within in a container?
- How components interact each other ?
- How the software works at a high-level?



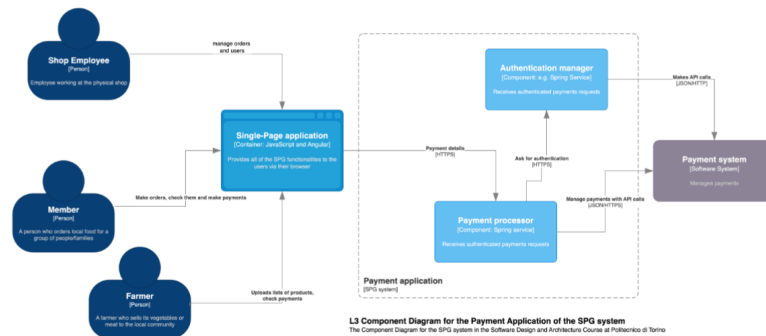
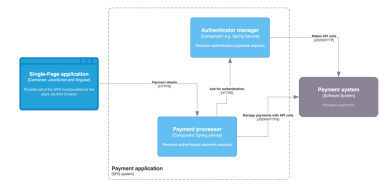
Optional, especially for containers like data storage, microservices. It is for technical people within the software development team.

6.7.1 Elements

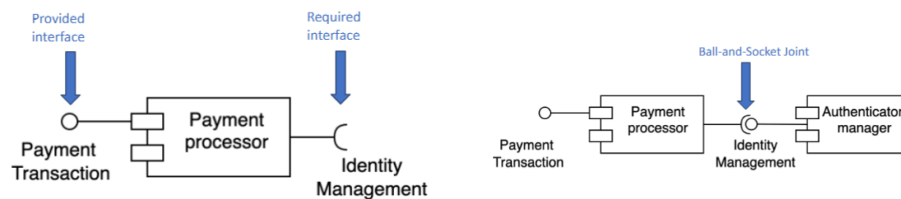
Same of the containers + components: people, software systems, containers and components.

Information to show for each component:

- Name: The name of the component.
- Technology: The implementation technology for the component (e.g. Java, Object, C#, Ruby).
- Description: A short description of the component in terms of its responsibilities



The UML component diagram is useful to provide more detailed documentation.



Ports

Ports indicate that required/provided interfaces are delegated to an internal class.



Usually, UML class diagrams are used beacuse:

- [illegible]

We can use Draw.io but no check on syntax. Instead we can use <https://academic.signavio.com/p/login>.

6.9 Summary of C4 model

Four main elements, associated with unidirectional lines, directions shown is the most relevant.

- line styles for synchronous/asynchronous communication;
- colors for different components;
- different shapes for specific types of components;

Capitolo 7

Wring Components

7.1 Components vs. Service

- **Component:** reusable piece of software that is used locally, eg static/dynamic linking. Is out of control of the writer of the component.
- **Service:** reusable piece of software that is typically accessed remotely, either synchronously or asynchronously, eg web service, RPC or socket.

7.2 Framework

A framework is a library that provides basic reusable assets that can be extended by the developers. Instead of writing from scratch, use some code that already exists.

It provides an abstraction layer over lower-level code, allowing developers to focus on higher-level features.

Note: Extension and customization are often based on the inversion of control.

7.2.1 Inversion of control

"Don't call us, we'll call you"

With inversion of control, your code does not control the overall flow of the program. Instead, a framework or container controls execution and calls your code at the appropriate time.

Example of direct control

```
void main() throws IOException{
    Path path = Path.of("data/grades.csv");
    List<String> lines =
        ↪ Files.readAllLines(path);
    List<String> h =
        ↪ Arrays.asList(lines.get(0).split(","));
    processHeaders(h);
    lines.stream().skip(1)
        .map(r -> Arrays.asList(r.split(",")))
        .forEach(r -> processRow(r));

    double average = sum / (lines.size()-1);
    IO.println(average);
}
```

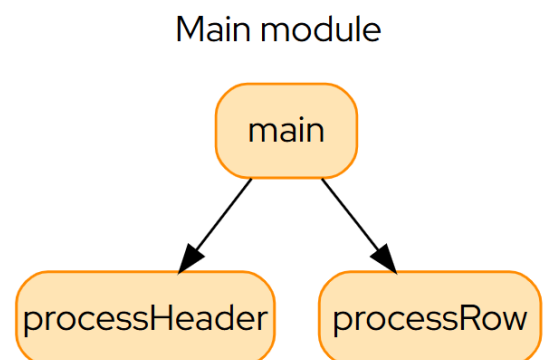


Figura 7.1: Call diagram

Example of inverted control

```
void main() throws IOException{
    CsvReader reader = new CsvReader();
    reader.headerProc(ReadCsvIoC: :
        ↪ processHeaders);

    reader.rowProc(ReadCsvIoC:
        ↪ :processRow);

    int n = reader.read("data/grades.csv");
    IO.println(sum/n);
}
```

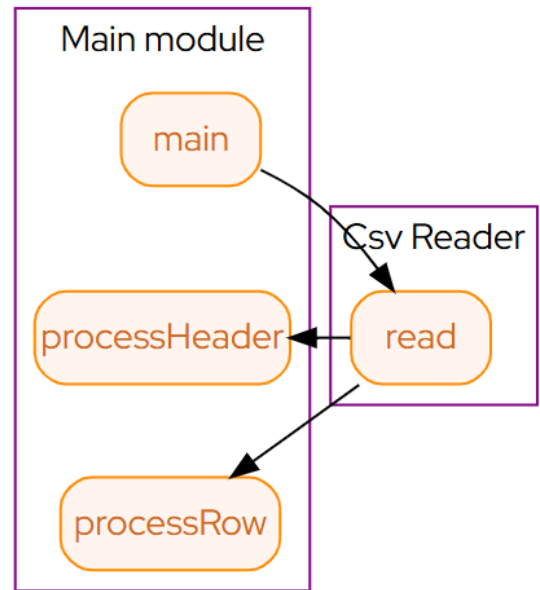


Figura 7.2: Call diagram

7.2.2 Patterns that enable inversion of control

The **Template method**, the **Strategy pattern**, and the **Observer pattern** are design patterns that enable inversion of control.

7.3 Wiring components

7.3.1 Interacction mode

- **Call-based:** function or method calls, the base logic of lots programming languages.
- **Event-based:** messages are sent and received. Used in GUI and web applications.
- **state-based:** state is updated and read from a shared memory. Used in embedded systems, writing in a specific memory address, and some peripheral devices read it.

7.3.2 Call binding

- **Source inclusion:** The code of the component is included directly in the main program during compilation. Strong implementation and compile-time dependencies.
- **Static linking:** Strong implementation dependency, limited compile time.
- **Dynamic linking:** Limited implementation dependency, limited compile time
- **Dynamic loading:** Strong construction dependency, no compile time

7.3.3 Method/function calls

Problem of simple calls:

- Structural dependencies
- Runtime dependencies: spacial, temporal and synchronization dependencies

- Cascading failures
- Limited scalability
- Hard to evolve independently

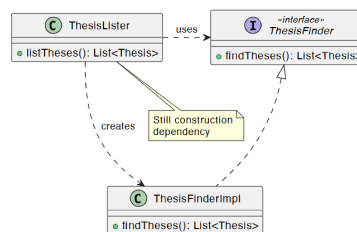
How put toghere components while minimizing the dependencies? E.g. they are build by different teams with little knowledge of each other.

Example A *ThesisLister*, frontend callses, provides a list of thesis matching a keykords using *ThesisFinder*.



Is not testable, not replaceble, heavy dependencies.

Using **interface** instead of concrete implementation to decouple the components, but still have dependencies.



7.4 Decoupling solutions

To solve the problem of dependencies, we can use 3 techniques for decoupling components :

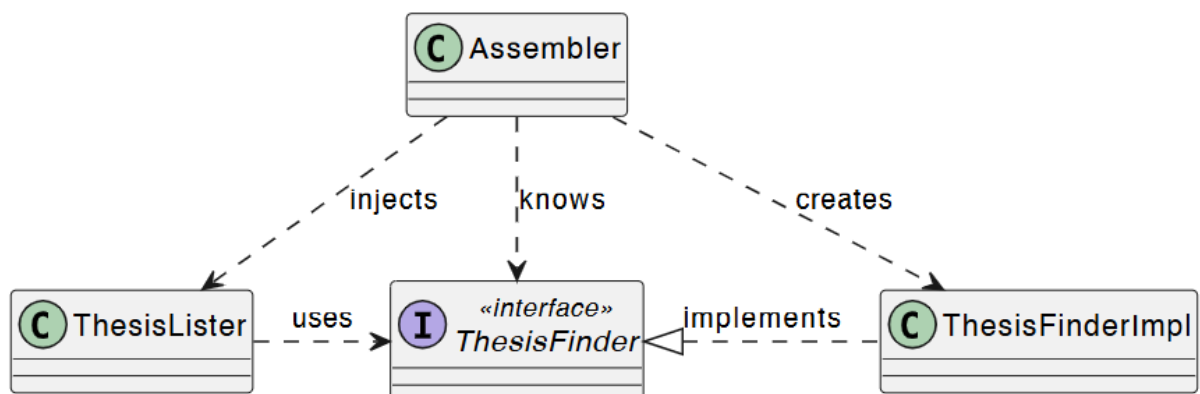
Dependency injection, Service locator, Message

7.4.1 Dependency injection

The *ThesisLister* calasss id dependent on both the *ThesisFinder* interace and the implementation of *ThesisFinderImpl*.

The goal of dependency injection is make the lister class unaware of the implementation class, but can still talk to an instance to do its work.

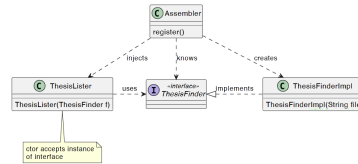
Move the logic of creating into an *Assembler* class



How to performe the injection? Two tecniques:

- **Constructor injection:** use the constructor of the dependent class to inject the dependencies.
- **Setter injection:** objects are initialized with dependency using a setter

7.4.2 Constructor injection



Basic injection

```

ThesisFinder finder = new ThesisFinderImpl(`theses.csv`); //creates the implementation with
↳ parameters
ThesisListener lister = new ThesisListener(finder); //injects the dependency upon construction
  
```

Constructor injection with container

```

Container c = new Container();
Object[] parameters = {"theses.csv"}; //parameters for the implementation ctor
c.registerImplementation(ThesisFinder.class, ThesisFinderImpl.class, parameters); // binding
↳ interface to implementation
c.registerComponent(ThesisListener.class); // component registration
  
```

Reflection is used to find out how to link.

Constructor injection with annotations

```

@Component
public class ThesisFinderImpl implements ThesisFinder {
    public ThesisFinderImpl(@Value("${thesis.file.path}") String file) {
        this.file = file;
    }
}

@Component
public class ThesisListener {
    public ThesisListener(ThesisFinder thesisFinder) {
        this.thesisFinder = thesisFinder;
    }
    //...
}
  
```

Note: The Spring framework is based on constructor injection with annotations.

7.4.3 Setter injection

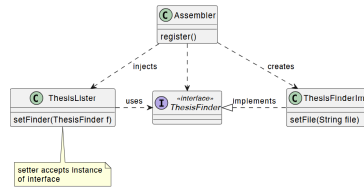
Instead of using the constructor we can use a setter method to inject the dependencies.

Basic injection

```

ThesisFinderImpl finder = new ThesisFinderImpl(); //creates the server object
finder.setFile("theses.csv"); //set the parameter
ThesisListener lister = new ThesisListener() //creates the client
lister.setFinder(finder); //injects the dependency with the setter
  
```

To set up a setting injection we can use a configuration file.



Setter injection with configuration file

```

<beans>
  <bean id="ThesisLister" class="it.polito.
    ↳ ThesisLister">
    <property name="finder">
      <ref local="ThesisFinder"/>
    </property>
  </bean>
  <bean id="ThesisFinder" class="it.polito.
    ↳ ThesisFinder">
    <property name="file">
      <value>theses.csv</value>
    </property>
  </bean>
</beans>
  
```

```

[
  {
    "id": "ThesisLister",
    "class": "it.polito. ThesisLister",
    "properties": [
      {
        "name": "finder",
        ↳ "local": "ThesisFinder"
      }
    ]
  },
  {
    "id": "ThesisFinder",
    "class": "it.polito.
    ↳ ThesisFinderImpl",
    "properties": [
      {
        "name": "file",
        ↳ "value": "theses.csv"
      }
    ]
  }
]
  
```

Setter injection with annotations

```

@Component
public class ThesisLister {

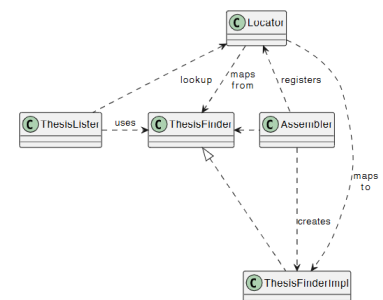
  private ThesisFinder finder;

  public ThesisLister() {}

  @Autowired
  public void setThesisFinder(ThesisFinder thesisFinder) {
    this. thesisFinder = thesisFinder;
  }
}
  
```

7.5 Service locator

A service locator is an object that knows how to get hold of all of the services that an application might need. It is an alternative to dependency injection to break the dependencies.



Example of locator service

Criterion	Constructor	Setter
Mandatory	Yes (object creation)	No (later stage)
Validity	Yes, always	Only after injection
Immutability	Yes (<code>final</code>)	No
Null safety	Fails fast	Risk later <code>NullPointerException</code>
Understandability	Depends explicit ctor	Depends hidden in setters
Circular dependencies	No	Can sometimes resolve
Optional dependencies	No, requires <code>Optional<></code> or ctor overload	Yes (with <code>@Autowired(required = false)</code>)

```

public class ThesisLister {
    public static String FINDER_SVC="thesis-finder";
    private final ThesisFinder thesisFinder;

    public ThesisLister() {
        this.thesisFinder = Locator.getLocator().lookup(FINDER_SERVICE, ThesisFinder.class);
    }
    //...
}

```

Basic assembler

```

Locator l = Locator.getLocator();
l.register(ThesisLister.FINDER_SVC, new ThesisFinderImpl("theses.csv"));
ThesisLister thesisLister = new ThesisLister();

```

7.5.1 Locator vs. Dependency injection

Criterion	Service Locator	Dependency Injection
Decoupling	Yes	Yes
Mechanism	Client explicit lookup	Provided externally
Inversion of Control	No	Yes
Visibility of dependencies	Hidden inside method calls to locator	Explicit in constructor/setters
Dependency on infrastructure	Client depends on the locator	No dependency on injector/container API
Ease of debugging	Easier (explicit calls)	Harder (framework wiring)
Simplicity	Straightforward	Less obvious at first
Reusability	Limited	Yes, no assumptions
Deferred linking	Yes, call locator anytime	No, injected at startup

7.6 Messages/Events

Replace Direct call with a Broker



In this case sender does NOT know:

- Who consumes the message
- When the message is processed
- How many consumers exist

7.6.1 Roles: Producer, Consumer

Producer	Consumer
Sends messages	Receives messages
Does not wait for processing	Processes independently
Does not know consumers	Can scale horizontally

7.6.2 Roles: Broker

The broker, like in real life, is an intermediary that facilitates the communication between producers and consumers. It's a middleware responsible for:

- Storing messages
- Delivering messages to consumers
- Handling routing and filtering of messages

Some examples of brokers are RabbitMQ, ActiveMQ, Kafka, Amazon SQS.

7.6.3 Delivery Guarantees

It is possible to configure different levels of delivery guarantees depending on the requirements of the application.

Type	Meaning
At most once	messages may be lost
At least once	messages may be duplicated, need idempotency
Exactly once	delivery guarantees, expensive

Distributed systems always involve trade-offs.

Producer Example

```
ConnectionFactory factory = new ConnectionFactory();
factory.setHost("localhost");

try (Connection connection = factory.newConnection();
    Channel channel = connection.createChannel() ) {

    channel.queueDeclare("orders", false, false, false, null);

    String message = "New Order: 123";

    channel.basicPublish("", "orders", null, message.getBytes());
    System.out.println("Sent: " + message);
}
```

Producer Example

```
ConnectionFactory factory = new ConnectionFactory();
factory.setHost("localhost");

Connection connection = factory.newConnection();
Channel channel = connection.createChannel();

channel.queueDeclare("orders", false, false, false, null);

DeliverCallback callback = (tag, delivery) -> {
    String message = new String(delivery.getBody());
    System.out.println("Received: " + message);
};

channel.basicConsume("orders", true, callback, tag -> {});
```

7.6.4 Structure

Producer and consumer are independent processes, they can run on different machines and the broker is infrastructure.

7.6.5 Shared state example

Queue	Topic
1 message → 1 consumer	1 message → many consumers
Work distribution	Event broadcast
Load balancing	Publish/subscribe

7.6.6 Effects

- **Construction decoupling:** producer does not create consumer
- **Spatial decoupling:** producer does not know location of consumer
- **Temporal decoupling:** consumer does not have to be active with producer. Producer does not have to block waiting for an answer
- **Data dependency:** data format and schema must be shared

7.6.7 Challenges

- Difficult debugging
- Eventual consistency
- Duplicate handling (idempotency!)
- Ordering issues
- Schema evolution

7.7 Shared state

Is a database just storage, or is it a communication mechanism?

Communication via mutations of shared, persistent state. The components communicate by reading and writing a common data store, no explicit message passing.

Shared state examples

- In-memory global variables
- Shared files
- Shared database
- Distributed cache (Redis)

Communication patterns

- State-based signaling
- Polling
- Work queue
- Derived state sharing

7.7.1 Effects

- No structural dependencies
- Low temporal dependencies: DB is like message broker
- Very high data dependency
- Simple mental model
- Easy querying and debugging

Aspect	Shared DB	In-memory Cache (Redis)
Nature	Persistent system of record	Ephemeral shared memory
Latency	Higher	Very low
Consistency	Strong	Config-dependent
Visibility	High (queryable history)	Low (state disappears)

Capitolo 8

Communication

The communication between components can be implemented in different ways:

- Local vs Distributed
- Synchronous vs Asynchronous
- One-to-one vs One-to-many
- One-way vs Two-way
- Call vs Message vs Shared memory

There are some distributed computing fallacies that are worth mentioning:

1. The network is reliable;
2. Latency is zero;
3. Bandwidth is infinite;
4. The network is secure;
5. Topology doesn't change;
6. There is one administrator;
7. Transport cost is zero;
8. The network is homogeneous.

Network latency - Due to decomposition an operation may result into a series of round-trips between two services. Some solution can be: Batch API that allow feeding multiple requests or merge services to convert messages to method calls,

	Multiplicity	
	One-to-one	One-to-many
Sync	Request/response	–
Async One-way	Notification	Publish/Subscribe
Async Two-way	Async. Req./Resp.	Publish/Async. Resp.

8.0.1 One-to-one communication

Request-response - A client makes a request to a service and waits for a response. The client blocks while waiting.

Asynchronous request/response - A client sends a request to a service, which replies asynchronously. The client doesn't block while waiting.

One-way notifications - A client sends a request to a service, but no reply is expected or sent.

8.0.2 One-to-many communication

Publish/subscribe - A client publishes a notification message, which is consumed by zero or more interested services.

Publish/async. response - A client publishes a request message and then waits for a certain amount of time for responses from interested services.

8.1 Messaging

Messages are exchanged over message channels. A sender writes a message to a channel, and a receiver reads messages from a channel. The communication is asynchronous, the sender doesn't block waiting for a reply

Header The header contains metadata about the message, such as:

- Message ID
- Return address
- Metadata that describes the content of the message
- Collection of key-value pairs useful for routing, filtering, etc.

Body The body contains the data being sent in either text or binary format.

8.1.1 Message types

Command - This is equivalent of an RPC request. It specifies the operation to invoke and its parameters.

Document - Contains only data. The receiver decides how to interpret it, e.g. a reply to a command

Event - Indicates that something notable has occurred. An event is often a domain event, which represents a state change of a domain object.

8.2 Channel

Abstraction of the mechanism to transfer messages.

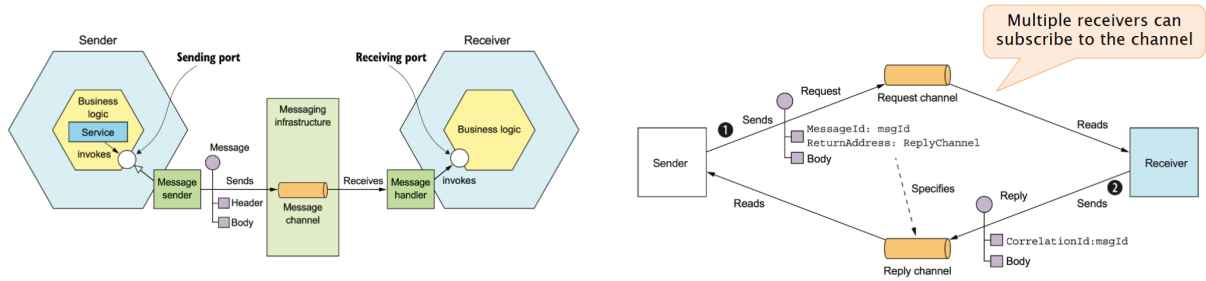
8.2.1 Point-to-point channels

Delivers a message to exactly one of the consumers that is reading from the channel.

Services use point-to-point channels for the one- to-one interaction styles. E.g. command messages

8.2.2 Publish-subscribe channels

Channel delivers each message to all of the attached consumers. Services use publish-subscribe channels for the one-to-many interaction styles



8.3 Supported styles

- Request/response - supports synchronous or asynchronous
- One-way notification - No response
- Publish/subscribe - Multiple subscribers no response
- Publish/Async responses - Multiple subscribers, multiple responses

8.4 Direct messaging - brokerless

Pros	Cons
Lighter traffic load, less latency	Services must be known <i>to</i> discovery
No centralized point of failure	Reduced availability, both must be available at the same time
Simplicity	Hard to have guaranteed delivery

8.5 Broker-based messaging

Broker is an intermediary components through which all messages flow.

Pros	Cons
Loose coupling. No need to know each other or to be active at the same time	Potential performance bottleneck
	Single point of failure
	Additional complexity

8.5.1 Broker characteristics

It support both standard protocols, e.g. AMQP, MQTT, STOMP, or proprietary protocols. It has some good feature such as:

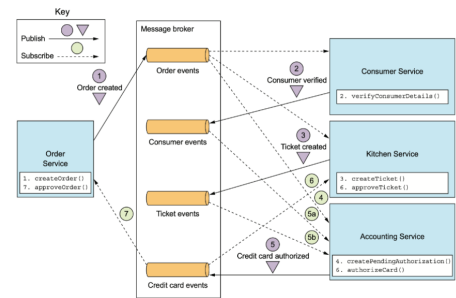
- Messaging ordering
- Guaranteed delivery
- Persistence
- Durability, consumer can disconnect and reconnect without losing messages
- Scalability
- End-to-end latency
- Support for cometing consumers

8.6 Coordination

8.6.1 Choreography

Each local transaction publishes domain events that trigger local transactions in other services.

- It requires reliable message infrastructure and messages with correlation id.
- Basic operation is simple, it perform operation and publish event
- Difficult to understand and maintain because transaction logic is scattered
- Complex service dependencies also cyclic, might lead to tight coupling

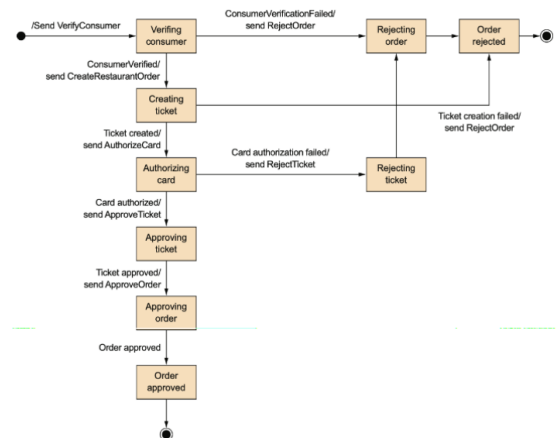
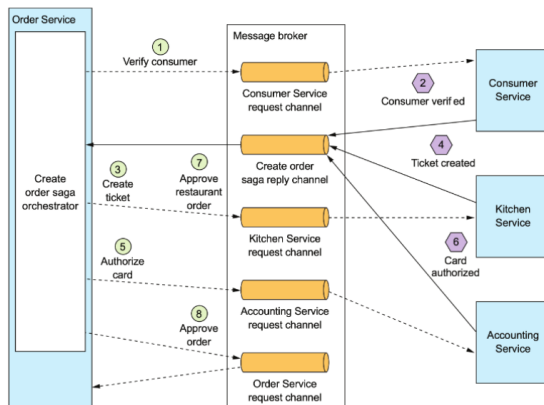


8.6.2 Orchestration

An orchestrator (object) tells the participants what local transactions to execute.

This technique requires reliable messaging, it has a simple dependency structure with reduced coupling.

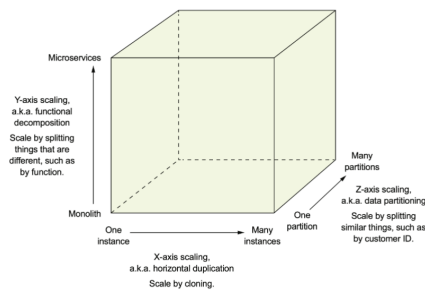
The business logic is simple and services are not aware of the saga. Transaction logic is gathered in a single place, must be kept separate from service logic.



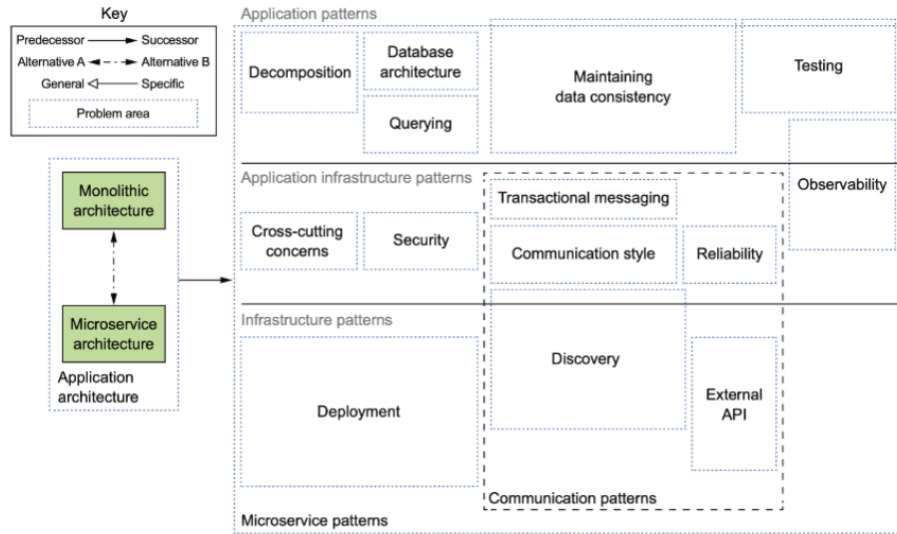
Capitolo 9

Microservices

The scalability dimension of software architecture can be represented by the following figure:



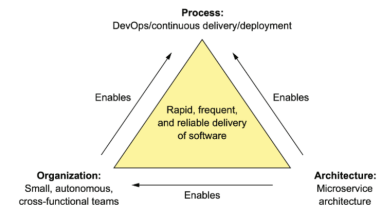
Advantages	Disadvantages
Enables the continuous delivery and deployment of large, complex applications	Finding the right set of services is challenging
Services are small and easily maintained	Distributed systems are complex, which makes development, testing, and deployment difficult
Services are independently deployable	Deploying features that span multiple services requires careful coordination
Services are independently scalable	Deciding when to adopt the microservice architecture is difficult
Enables teams to be autonomous	
Allows easy experimenting and adoption of new technologies	
Better fault isolation.	



9.0.1 Monolith vs. Microservices

The problem is the deployment in fact:

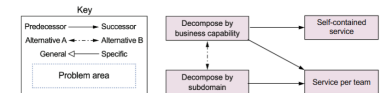
- Monolithic architecture: Simple to develop, deploy, and scale.
- Microservices architecture: Simple to maintain, to test and flexible to scale



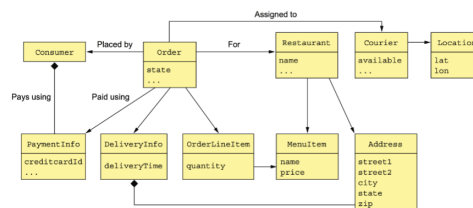
9.0.2 Decomposition

Decomposition identifies system operations:

- Understand domain model
- Define operations
- Identify use cases



and decompose the system into subsystems or subdomains.



Domain model Example

User stories

By business capability

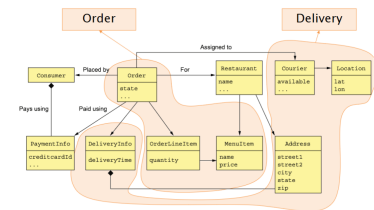
1. An organization business capability is what it does (as opposed to how). Look into organization structure, also consistent operations, stories or goals.
2. Single Responsibility Principle

Actor	Goal	Services
Consumer	Create order	Order
Restaurant	Accept order	Order
Restaurant	Complete order	Kitchen
Courier	Update location	Delivery
Courier	Pick up	Delivery
Courier	Delivery	Delivery
Delivery company	Manage couriers	Courier
Delivery company	Courier payment	Accounting

3. Common Closure Principle - classes that change for the same reason should be in the same package
4. Result is cohesive and loosely coupled

By subdomain

- Identify bounded contexts - DDD sub-domains. Partitions of overall domain model
- Result is cohesive, Loosely coupled, allow cross-functional and autonomous teams and scalable architecture



9.0.3 Guidelines from OOP

Single Responsibility Principle - A class should have only one reason to change.

Common Closure Principle - The classes in a package should be closed together against the same kinds of changes. A change that affects a package affects all the classes in that package.

Decomposition obstacles

- Network latency
- Reduced availability due to synchronous communication
- Data consistency across services
- Obtaining a consistent view of the data
- Good classes

9.1 Business logic

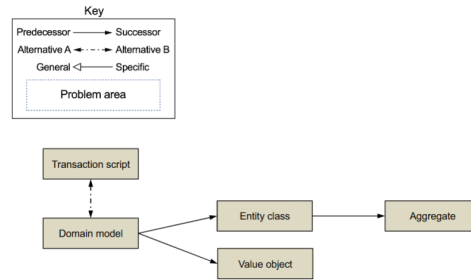
9.1.1 Transaction script

When you have simple business logic that consist of a sequence of steps.

Organize business logic by procedures where each procedure handles a single request from the presentation.

One service class with a method for each procedure. Several data classes.

Consequences:



- All the code for a request in a single place
- Code duplication for similar requests
- Difficult to maintain and refactor

9.1.2 Domain model

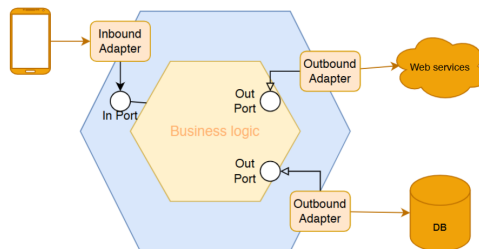
When you have complex business logic

Adopt recommended object-oriented methods to create classes that encapsulate both code and data, using the DDD approach.

Consequences:

- Code for a request is spread among classes
- No code duplication
- Easy to maintain and refactor
- Easy to extend with Strategy or Template Method patterns

9.1.3 Ports & Adapters (hex)



9.2 Domain Driven Design - DDD

- Sub-domains
- Bounded context
- Class types patterns: Entity, Value Object, Service, Repository, Factory
- Aggregate

9.2.1 Sub-Domain & Bounded Context

DDD decompose the overall domain model into sub-domains. Each sub-domain is valid within a bounded-context.

A bounded-context brings together a common language and people, team, focusing on a shared problem. Distinct contexts are related to each other in order to enable communication.

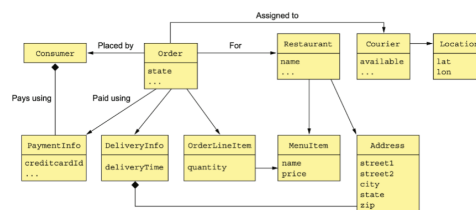
9.2.2 Class patterns

- Entity - Represents objects that have a persistent identity. Two objects with the same values represent distinct items. Usually, they are persisted to a db
- Value object - Represents a collection of values. Two objects with the same values are interchangeable
- Factory - Implement object creation logic. May hide concrete class
- Repository - Provides access to persistent entities. Hides access to db
- Service - Implements business logic that does not belong to a single entity

9.2.3 Domain boundaries

Often domain models have fuzzy boundaries. All strictly related concepts should belong to the same context. E.g. `Order` and `OrderLineItem` classes.

Not keeping together such classes might result in violations of business rules or invariants, upon updates. E.g. minimum amount for an order.

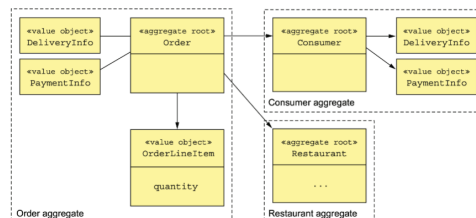


9.2.4 Aggregate pattern

Organize a domain model as a collection of aggregates, each of which is a graph of objects that can be treated as a unit.

Decompose domain model into chunks:

- Self contained units
- Easier to understand
- Clarify scope of operations
- All elements share the same life cycle



9.2.5 Rule: reference roots only

Reference only the aggregate root. Updates go through methods of aggregate root. When a repository loads an aggregate from the db it returns a reference to the aggregate root.

Consequences:

- Enable aggregate to enforce consistency rules

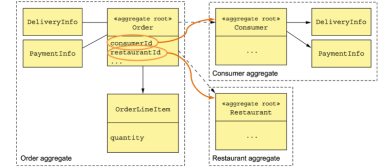
- Aggregate updates are serialized

9.2.6 Rule: primary keys inter aggregate

Inter-aggregate references **must use primary keys**. NOT using object reference, indeed, this is usually code smell.

Consequences:

- Loose coupling of aggregates
- Keep aggregate boundaries enforced
- Avoid accidental cross-aggregate updates
- Aggregates can belong to distinct services
- Simplify persistence

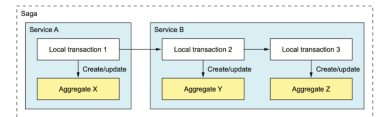


9.2.7 Rule: one aggregate per transaction

One transaction operates on one aggregate only

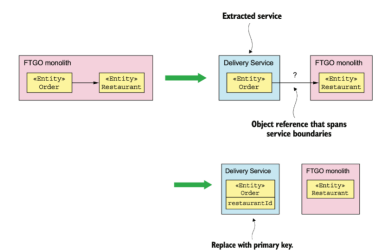
Consequences:

- Ensure a transaction is contained in a single service
- Suitable for limited transactions support in NoSQL databases
- Less efficient in transfer from/to db
- More complex implementation of complex requests that update multiple aggregates



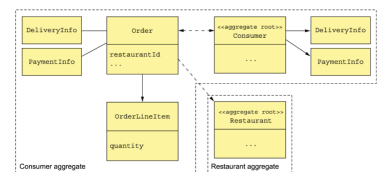
9.3 Split monolith with aggregates

Transformation into aggregates can help splitting the monolith. Services can be extracted easily.



9.3.1 Aggregate granularity

- Fine grained: Improve scalability (Updates on an aggregate are serialized), reduce the risk of conflicts and it is easier to decompose into services
- Coarse grained: More suitable to fit all relevant entities in a transaction



9.3.2 Domain Events

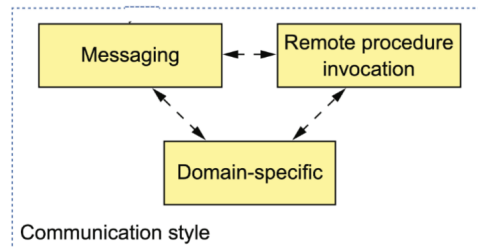
An Aggregate publishes a Domain Event when it's created or undergoes some other significant change. Represented by a class in the domain model, the name is formed using a past-participle verb phrase. E.g. **OrderPlaced**, **OrderShipped**, etc.

Properties that convey the event meaning. Each property is either a primitive value or a value object, also metadata, e.g., event ID, timestamp

- Maintaining data consistency across services
- Notifying a service that maintains a replica that the source data has changed
- Sending notifications to users

- Notifying another service to update related data
- Notifying a different application via a registered webhook or via a message broker
- Monitoring domain events to verify that the application is behaving correctly.
- Analyzing events to model user behavior

9.4 Communication



Ensure agreement on service interaction mechanism to avoid late integration issues:

- Define the interface (using an IDL)
- Review with client services' developers
- Iterate to reach consensus
- Start implementation of services

9.4.1 API evolution

"An interactive software system must be continually adapted or it becomes progressively less satisfactory".

Aim for minor backward-compatible changes.

Robustness principle:

- Be conservative in what you do
- Be liberal in what you accept from others

9.5 Message formats

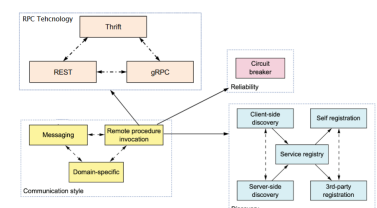
Text-based formats: JSON, XML, YAML

Binary formats: Protocol Buffers, Avro, Thrift

Once the format is chosen, it might be mandated by the communication mechanism

9.6 Remote procedure call (RPC)

- **Pros:** **Simple** and familiar, request-response is easy and the system is simple because of no intermediate broker
- **Cons:** Usually **only supports request/reply** and not other interaction patterns such as notifications, request/async response, publish/subscribe, publish/async response.
Reduced availability since the client and the service must be available for the duration of the interaction



Requires service discovery

9.6.1 Robust Synchronous APIs

- Network timeouts: Never block indefinitely, always use timeouts when waiting for a response
- Limit outstanding requests: Define an upper bound on the number of outstanding requests that a client can make to a particular service
- Circuit breaker pattern, allow failing fast

9.6.2 Circuit Breaker pattern

Service should be invoked through a proxy. Track the number of successful and failed requests. If the error rate exceeds a threshold, trip the circuit breaker so that further attempts fail immediately. After a timeout period, the client should try again, and, if successful, close the circuit breaker.

This is implemented by Netflix's Hystrix library.

9.6.3 Service Discovery

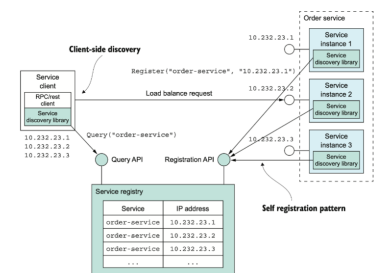
Using RPC you need to know the network location of your service.

In microservices architectures service instances change continuously due to:

- Autoscaling
- Failures
- Updates

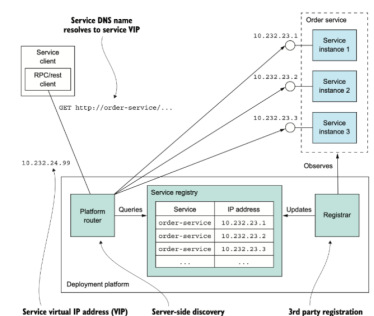
Application level discovery

- Self registration pattern: service register itself on activation and update list (Health check endpoint on server, Heartbeat API callback on registry)
- Client-side discovery pattern: Queries registry for active servers. Uses load balancing algorithm to pick one.



Platform level discovery

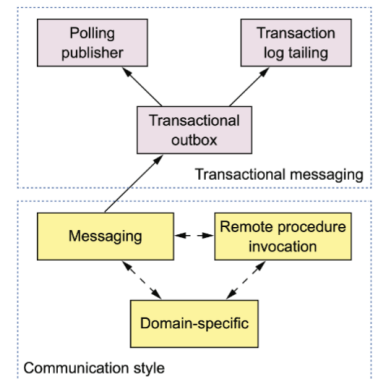
- Self registration pattern
- Third party registration pattern: Registrar component perform registration, often part of the platform (e.g., Kubernetes or Docker)
- Server-side discovery pattern: Client makes DNS request. Router access registry, load balance and resolve DNS



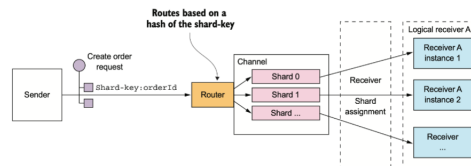
9.7 Messaging

9.7.1 Competing receivers

- Multiple receivers
- Each message must be processed only once or processed in order
- Sharded (partitioned) channels. The sender specifies a shard key in the message's header. The message broker uses a shard key to assign the message to a particular shard/partition.



9.8 Shared channels



9.8.1 Duplicate messages

Failures of client, server, or broker might represent a problem. Message are guaranteed to be delivered at least once, but not only once. Possible countermeasures:

- Idempotent messages
- Message tracking

9.8.2 Transactional messaging

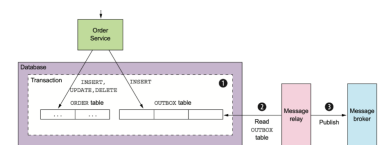
Use a db table as a message queue. Keep in the same transaction:

- Db application update
- Message queueing

Transactional outbox

The outbox table acts as a message queue within the database.

1. The transaction updates the order table and inserts a message into the outbox table.
2. A message relay reads the outbox table.
3. The relay publishes the message to the message broker.

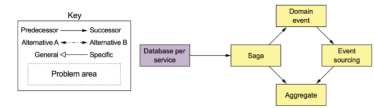


9.9 Data and Transactions

9.9.1 Database per service

Services must be loosely coupled so that they can be developed, deployed and scaled independently. Each service's persistent data is private and accessible only via its API, options:

- Private-tables-per-service
- Schema-per-service
- Database-server-per-service



A service's transactions only involve its db.

9.9.2 Transactions in a monolith

ACID db transaction : Atomicity, Consistency, Isolation, Durability. Lack of isolation leads to:

- Lost updates
- Dirty reads
- Nonrepeatable reads

Example: confirm order

- **Order Service:** Create an Order in an APPROVAL_PENDING state.
- **Consumer Service:** Verify that the consumer can place an order.
- **Kitchen Service:** Validate order details and create a Ticket in the CREATE_PENDING state.
- **Accounting Service:** Authorize consumer's credit card.
- **Kitchen Service:** Change the state of the Ticket to AWAITING_ACCEPTANCE state.
- **Order Service:** Change the state of the Order to APPROVED state.

9.9.3 Distributed data consistency

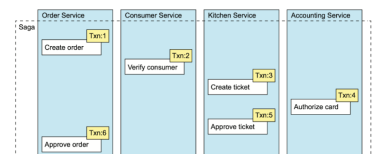
- **Distributed transaction - 2PC:** Query to commit, Agree/Abort, Commit/Rollback, Acknowledge.
- **Saga:** Local transactions coordinated with messages.

Distributed transactions limits

Transactions are not universally supported (e.g. NoSQL dbs, Modern message brokers). Transactions are synchronous, which means reduced availability. Isolation is overrated (most dbs adopt weaker isolation levels). Impossible to have both availability and most recent data.

9.9.4 Saga pattern

A saga is a sequence of local transactions. Each local transaction updates the database and publishes a message or event to trigger the next local transaction in the saga. If a local transaction fails then the saga executes a series of compensating transactions (undo the changes that were made by the preceding local transactions).



Compensating transactions

Rollback is not directly supported since all service transactions are separate. Explicit invocation of compensating transactions in case of rollback (Used, e.g., when a check in a step fails).

Saga transactions types

- **Compensatable transaction:** Can potentially be rolled back using a compensating transaction.
- **Pivot transaction:** The go/no-go point in a saga. If succeeds the saga can run until completion.
- **Retriable transaction:** Follow the pivot transaction and is guaranteed to succeed.

Saga non-isolation

- **Atomic:** Managed by coordination mechanism and compensating transactions.
- **Consistent:** each service guarantee consistency in its own transaction.
- **Durable:** ensured by local db.

Non isolated problems: Lost updates, Dirty reads, Non-repeatable reads.

Countermeasures for non-isolation

- **Semantic lock:** An application-level lock. A compensatable transaction sets a flag in any record that it creates or updates indicating it isn't fully committed. It's cleared by either a retrieable transaction or a compensating transaction.
- **Commutative updates:** Design update operations to be executable in any order (e.g., `Account.debit()` decrement balance).
- **Pessimistic view:** Reorder the steps of a saga to minimize business risk due to a dirty read. Cancel order (rolled back) and create order when executed concurrently could lead to exceeding customer's credit.
- **Reread value:** Prevent dirty writes by rereading data to verify that it's unchanged before overwriting it. If changed, saga aborts.
- **Version file:** Record the updates to a record so that they can be reordered (turns noncommutative operations into commutative).
- **By value:** Use each request's business risk to dynamically select the concurrency mechanism (sagas vs distributed transactions).

9.9.5 Domain event pattern

An aggregate publishes a domain event when it's created or undergoes some other significant change. A domain event is a class with properties that meaningfully convey the event, a name formed using a past-participle verb (e.g. `OrderCreated`), and metadata (e.g. event ID, timestamp).

9.9.6 Event sourcing

Event sourcing is an event-centric technique for implementing business logic and persisting aggregates. An aggregate is stored in the database as a series of events, where each event represents a state change. It is an alternative to common persistence, e.g. through ORM.

Pros	Cons
Reliably publishes domain events	It has a different programming model that has a learning curve
Preserves the history of aggregates	Messaging-based application are complex
Avoids the O/R impedance mismatch	Evolving events can be tricky
Provides developers with a time machine	Deleting data is tricky
	Querying the event store is challenging

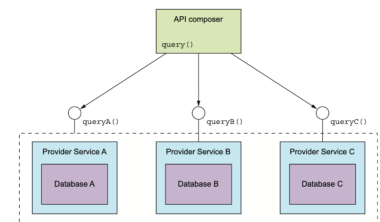
9.10 Queries

Queries can be Simple (require data from a single μ S db) or Composed (required data from multiple μ S dbs).

9.10.1 API Composition

- **API Composer:** Implement a query that retrieves data from several services by querying each service via its API and combining the results.
- **Provider Service:** Implement a service-limited query, limited by how data is partitioned and capabilities of APIs/databases.

Composer role can be Client (e.g. web app backend), API Gateway, or a Standalone service. For efficiency, parallelize queries as much as possible.

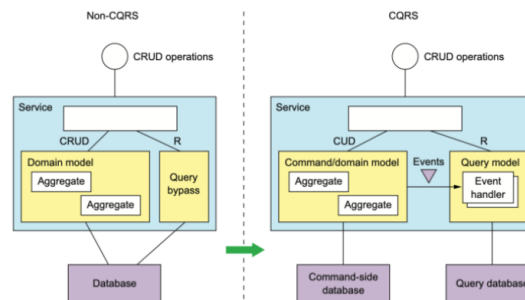
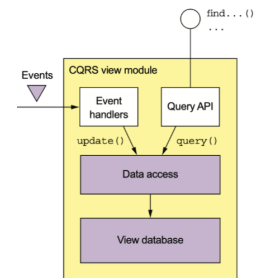


9.10.2 CQRS

Command - Query Responsibility Segregation. Use events to maintain a read-only view that replicates data from the services. The view can be easily queried.

Design the schema to match the queries. The data access module must handle idempotent updates and concurrent updates.

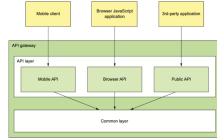
Implement a mechanism to efficiently build or rebuild the view, and cope with the replication lag.



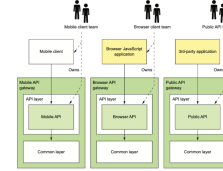
9.11 External API

External clients have low bandwidth (negative impact on UX) and lack of encapsulation. Different communication mechanisms are used. Possible solutions:

Edge Functions Authentication, Authorization, Rate limiting, Caching, Metrics collection, Request logging.



Backend for Frontend (BFF)



9.12 Transition to Microservices

- **Big-bang rewrite:** "the only thing a Big Bang rewrite guarantees is a Big Bang!" (M.Fowler) .
- **Strangling the monolith:** Progressively wrap the monolith until it becomes an empty shell.

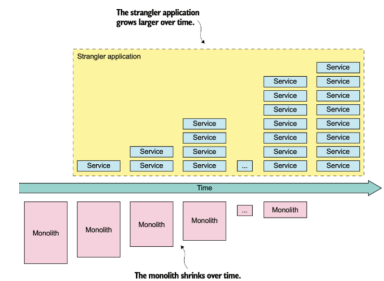
9.12.1 Strangling the monolith

Context: a monolith application should be migrated to a micro service one

Problem: a complete big-bang rewrite poses several risks

Solution: gradually create a new system around the edges of the old, letting it grow slowly over several years until the old system is strangled.

Consequences: Reduced risks, frequent releases allow to monitor the process and provide steady release of value, speed of transition can be adjusted.



9.12.2 Transition strategies

- **Implement new features as services:** Stop the growth of the monolith. API gateways reroute requests. Integration glue code integrates the service with the monolith (typically adapters).
- **Separate presentation and backend:** Represents an intermediate strategy.
- **Break up the monolith:** Extract functionalities into services. Extract a vertical slice (Inbound adapters, Domain logic, Outbound adapters, Monolith's database schema).

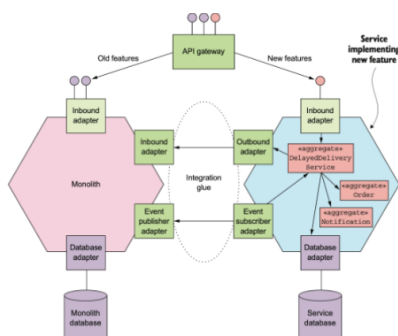


Figure 9.1: New features as services

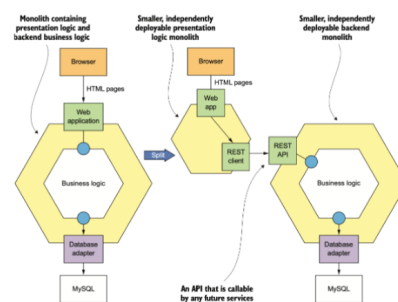


Figure 9.2: Split separation and backend

9.12.3 Refactor DB

Several classes are persistent so the relative tables in the db must be refactored. Replicate data to avoid widespread changes in the monolith (Keep original schema and use triggers to update data).

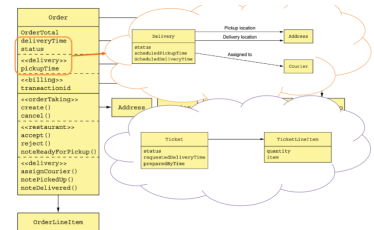
Selecting services to migrate Rank modules by anticipated value your organization can get by migrating:

- Features providing new business capabilities
- Accelerated development
- Performance, scaling, reliability issues
- Unlock migration of other more valuable modules

9.12.4 God Class

An object that controls way too many other objects in the system and has grown beyond all logic to become The Class That Does Everything. Often implements business logic for many different aspects.

Splitting the God Class: Apply DDD to separate domain models into subdomains. Avoid tight coupling among services and avoid a data service. Different views must be kept consistent (via Saga) and UX must be adapted to the set of partial models (via API gateway).



Capitolo 10

Bending Spoons seminary (Evernote)

Founded in 2013 in Danmark. They acquire digital products then unlock the potential of the products.
500+ TB data processed per year. Constantly search for new talent.

They look for:	The processes:
Drive	Screening
Potential	Tasks
Passion	Interviews
	Final decision

10.1 Evernote

There is a frontend that get the data from bakend at the very high level, but deeper the system is very complex and full of components.

Before the acquisition Evernote was a monolith, but after the acquisition they decided to migrate into a set of microservices with each one thake care of a specific element.

All of there service shoudl minimize the dependencies with the other services.

But the backend is not the complex part of the system.

10.1.1 Evernote's sync system

Legacy: "Monolith have you some new content?" "Yes, the note with the monkey" This is a polling request useless for the user and for the backend system.

Microservices: Persistent connection between the client and the server, faster then polling, cheaper for both: the client side save battery and for the server side save CPU and bandwidth.

Deep dive into sync activity

Note creation:

- The phone perform an http request to the backend to note service
- Performing some checks
- Create the json file SQL database
- Note create message to the sync service queue
- The sync service send the message to the all the client devices with the new note

Client connection

Fernando authenticate with the websocket and then connect to sync service. This service has tons of persistent connection always open, it uses a multi-map, because of multiple devices but same user, to keep track of the connection and the user id.

The persistent connection can go offline for a bit. Do they lose all the message? Yes but the sync service uses Google Cloud Datastore. It is a single schema with the UID, the Instance with every metadata of Fernando and SyncOperation with the operation performed (CREATE, UPDATE, DELETE).

Every time the client receives some message, it receives the userId and the timestamp and this is used in the Google Cloud Datastore.

```
SELECT * FROM sync_documents
WHERE accessInfo.updated >= "1259414" and accessInfo.userId = "12345"
```

10.1.2 The sync service

Pros	Cons
Fast updates	Data duplication
Offline access	Integration difficulty
Cheap	Complexity

Capitolo 11

Boundary

1. Partition the system into components

11.1 Independence and decoupling of layers

Capitolo 12

Monolithic Architectural Styles

12.1 Architectural Quanta

The architectural quanta is the smallest part of the system that runs independently. An architectural quanta establishes the scope for a set of architectural characteristics. It features:

- Independent deployment from other parts of the architecture
- High functional cohesion
- Low external implementation static coupling
- Synchronous communication with other quanta

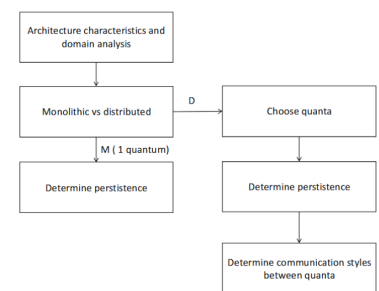


Figure 12.1: Scoping

12.2 Architectural styles

- Monolithic = single deployment unit of all code
- One architectural quantum
- Distributed = multiple deployment units connected through remote access protocols

Monolithic	Distributed
Layered architecture	Service-based
Pipeline architecture	Event-driven
Modular monolithic style	Space-based
Micro-kernel architecture	Service-oriented
	Microservices

12.3 Layered architecture - n tier

Also known as n-tier architecture. Technical partitioning. Each layer is independent and has a specific responsibility. A layer represents a function of the system. Pro: testability

Common patterns are:

- Presentation layer

- Business logic layer
- Persistence layer
- Database layer

In the layered architecture, the layers can communicate only with the adjacent layers. Teams needing structured development.

12.3.1 Layered architecture - variations



12.3.2 Adding layers

New layers to enforce architectural constraints, helps control access and dependencies. Example: add *service layer* for share common logic

12.3.3 Isolation

Each layer is independent, interacting only through defined contracts. Prevents tight coupling and cross-layer dependencies. The risk is the **Sinkhole antipattern**.

12.3.4 Open layers

Allow direct access to any layer for efficiency, useful for simple operations. Improves performance but reduces architectural control.

12.3.5 Open vs close layers

- Closed layers → stability, modularity, easier changes
- Open layers → flexibility, performance, but higher risk of coupling

Sinkholes acceptable threshold: 20% manageable, 80% change architecture style!

	Characteristic	Rating	Notes
Structural	Simplicity	Very High	
	Modularity	Very low	
Engineering	Maintainability	Very low	Changes can impact every layer
	Testability	Low	Mock/stub components
	Deployability	Very low	At every change !
	Evolvability	Very low	Changes can impact every layer
Operational	Responsiveness	Medium	Caching and multithreading
	Scalability	Very low	Lack of modularity
	Elasticity	Very low	Lack of modularity
	Fault tolerance	Very low	Faults impact the whole system, all layers must be restarted

When to use	When not to use
Applications with clear separation	Not ideal for high-performance or highly dynamic systems
Small, simple applications	Large and complex applications
As starting point for situations with low budget and short time	
To quickly test new ideas	
Examples of usage: operating systems, desktop applications	

12.4 Modular monolithic style

Domain partitioned: each module encapsulates a business capability. Strong internal boundaries required. Modules with clear responsibilities. Deployed as a single unit of software. E.g. Web archive (WAR) file.⁷

Modules might have their own dedicated repositories.

In this we decupled all the modules like Customer, Order, etc. The team is also oragnized in different way before a team for each layer, now we have a team for each modules or a pool of modules.

Data topology

Also in this style we can have single database or more domain-specific databases for the best performance.



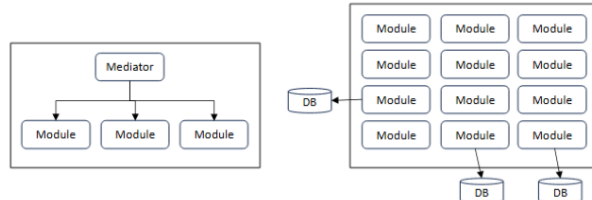
Interaction between modules

The modules **communicate directly** (peer to peer) or **through a mediator** the goal is orchestrate different modules.

12.4.1 Layered vs modular: components organization

- Layered: `com.app.presentation.customer.profile`
- Modular: `com.app.customer.profile`; `com.app.customer.profile.presentation`; ...

12.4.2 Variations of modular monolithic style



12.4.3 Advantages and disadvantages of modular monolithic style

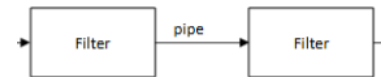
Key characteristics	Tradeoffs:
Simpler operational model than microservices	Risk of becoming tightly coupled over time
No network overhead	Limited independent scalability
Easier testing and debugging with respect to layered style	Large codebase can slow development
Requires discipline to maintain boundaries	Harder team autonomy at scale
	Risk of long start up

	Characteristic	Modular Monolith	Notes
Structural	Simplicity	Very High	
	Modularity	Low	
Engineering	Maintainability	Low	Better than Layered because of higher modularity
	Testability	Low	As above
	Deployability	Low	As above
	Evolvability	Low	As above
Operational	Responsiveness	Medium	
	Scalability	Very low	
	Elasticity	Very low	
	Fault tolerance	Very low	

When to use	When not to use
Early-stage systems needing structure	Systems with high demands for operational characteristics
Situations with tight time and budget constraints	Systems requiring independent scaling
Teams wanting simplicity over distribution	When majority of changes are technically-oriented
Teams working with DDD	
Applications with strong domain boundaries	

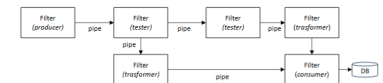
12.5 Pipeline architecture

Also known as Pipes and Filters. Technically partitioned. Data flows through a sequence of processing steps. Each step transforms input into output. Filters are independent and stateless (ideally)



12.5.1 Components

- **Filters:** processing units performing specific business functions
- **Pipes:** connectors passing data between filters, unidirectionally. Can be deployed as remote service (style remains monolith). Can be either synchronous or asynchronous (e.g., threads, messages).
- **Data format:** must be consistent or transformed
- **Database:** usually a single db, but filter can have their own database too
- **Producer:** starting point of a process
- **Transformer:** enhance, transform, compute (map)
- **Tester:** test input w.r.t. to criteria (reduce), optionally produces output
- **Consumer:** termination point, with persistence

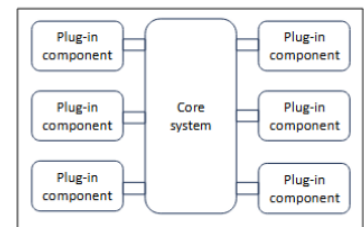


Key characteristics:	Tradeoffs:
Easy to extend by adding filters	Latency due to multiple stages
Clear separation of concerns	Harder debugging across pipeline
Examples: Bash, Zsh, MapReduce	Weak fit for complex business logic
	Data format coupling between filters

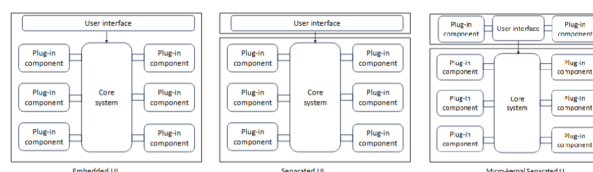
	Characteristic	Pipeline	Notes
Structural	Simplicity	High	
	Modularity	Low	Separation of concerns in filters
Engineering	Maintainability	Low	
	Testability	Medium	Separation of concerns in filters
	Deployability	Low	
	Evolvability	Medium	
Operational	Responsiveness	Medium	
	Scalability	Very low	Separate deployments can improve it, but at the expense of simplicity and cost.
	Elasticity	Very low	As above
	Fault tolerance	Very low	As above
When to use		When not to use	
Data processing and transformation pipelines		In systems with non-deterministic flows (use event-driven a.s.)	
Streaming or ETL systems		When fault tolerance is important	
If properly implemented: independent, reusable processing steps		When bi-directional communication is needed	
		If high demands of operational characteristics	

12.6 Micro-kernel - plug-in architecture style

Also known as Plug-in architecture. Core system with basic and stable functionalities («happy paths»). It can be technically or domain partitioned. Plug-ins extend system capabilities and incorporate complexity. Registry or discovery mechanism. Data topology: one db or plug-in specific data sources



12.6.1 UI: variants



12.6.2 Plug-in components

- They should be independent and self-contained
- Communication p2p with the core system, not between plugins
- Compile based vs runtime base
- Common semantic, e.g. `app.plugin.<domain>.<context>` -> Example. `app.plugin.editors.plain`
- Plug-in could a standalone remote service
 - The architecture remains monolithic due to coupling with core system
 - Advantages: scalability, throughput, asynchronous communication
 - Disadvantages: reliability, costs, complexity

Characteristics:	Trade-offs:
Highly extensible and customizable	Performance overhead from indirection
Separation between core and features	Complex plug-in lifecycle management
Supports product variability	Testing combinations of plugins can be hard
Stable core is critical	Core design mistakes are costly

When to use	When not to use
Common Product-based systems	Core systems that need frequent changes
Systems requiring extensibility	High dependencies between plugins
Platforms with plugin ecosystems	
Customization-driven applications	
Examples: browsers, IDEs, PMD, Jira	

12.7 Monolithic Styles - recap

	Characteristic	Microkernel	Notes
Structural	Simplicity	High	
	Modularity	Medium	Extensions easier with plugins
Engineering	Maintainability	Medium	Functionalities are better isolated w.r.t other monolithic styles
	Testability	Medium	As above
	Deployability	Medium	As above
	Evolvability	Medium	Extensions easier with plugins
Operational	Responsiveness	Medium	Microkernel applications are generally small
	Scalability	Very low	
	Elasticity	Very low	
	Fault tolerance	Very low	

	Characteristic	Layered (n-tier)	Modular Monolith	Pipeline	Microkernel
Structural	Simplicity	Very High	Very High	High	High
	Modularity	Very low	Low	Low	Medium
Engineering	Maintainability	Very low	Low	Low	Medium
	Testability	Low	Low	Medium	Medium
	Deployability	Very low	Low	Low	Medium
	Evolvability	Very low	Low	Medium	Medium
Operational	Responsiveness	Medium	Medium	Medium	Medium
	Scalability	Very low	Very low	Very low	Very low
	Elasticity	Very low	Very low	Very low	Very low
	Fault tolerance	Very low	Very low	Very low	Very low